Machine Learning

Rocky Sir's Lectures

CSE 4111 ? Machine Learning & Evolutionary Algorithms

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1	Lecture 01: Introduction to Machine Learning	Page 3
2	Lecture 02: Machine Learning Fundamentals	Page 16
3	Lecture 03: Genetic Algorithms (GA)	Page 60
4	Lecture 04: Swarm Intelligence & Particle Swarm Optimization (PSO)	Page 123
5	Lecture 05: Ant Colony Optimization (ACO)	Page 142
6	Lecture 06: Artificial Bee Colony (ABC)	Page 163

Machine Learning (ML)

- Machine Learning (ML) is a subset of artificial intelligence (AI) that enables systems to **learn from data and make predictions or decisions** without being explicitly programmed.
- It is about using algorithms that can **identify patterns in data and improve over time** with more data (experience).
- Example: A spam filter that learns to detect spam emails by analyzing features (contents, words, sender info, URLs, etc.) in previous spam and non-spam emails.

Basic Terminology in ML

Term	Description
Model	The mathematical representation of a real-world process learned from data.
Training	The process of feeding data to an algorithm to help it learn patterns.
Testing	Evaluating the trained model on unseen data.
Features	Input variables used to make predictions (e.g., height, age, keywords).
Labels	The desired output or target (e.g., spam or not spam).

Basic Terminology in ML (Cont.)

Term	Description
Algorithm	A method or process used to build the model from data.
Overfitting	When a model performs well on training data but poorly on new data.
Underfitting	When a model is too simple to learn the data patterns.
Supervised Learning	Learning from labeled data (input + output known).
Unsupervised Learning	Learning patterns from unlabeled data (no output).
Reinforcement Learning	Learning through rewards and penalties by interacting with an environment.

Deep Learning (DL)

- Deep Learning (DL) is a specialized subfield of machine learning that uses artificial neural networks, especially deep (multi-layered) networks.
- It is particularly powerful for large-scale and unstructured data like:

- Images
- Audio
- Text
- Video

Deep Learning (DL) (Cont.)

Use Deep Learning When:

1. You have a large dataset
Deep learning models need **a lot of data** to perform well. If you have **millions of samples**, deep learning can outperform classical methods.

Example: ImageNet with 14 million labeled images is ideal for training deep neural networks like CNNs.

Deep Learning (DL) (Cont.)

2. Your data is unstructured
•Images
•Audio/Speech
•Videos
•Natural Language (Text)

Examples:
•Image classification (cats vs. dogs) → Use CNN
•Speech recognition → Use RNN or Transformers
•Sentiment analysis or machine translation → Use LSTM or Transformers (e.g., BERT, GPT)

Deep Learning (DL) (Cont.)

3. You need automatic feature extraction
In traditional ML, **feature engineering** is done manually. Deep learning learns features directly from raw data using multiple layers.

Example: In facial recognition, CNNs automatically learn edges, shapes, and facial structures.

4. The problem is too complex for rule-based or simple ML models

Tasks like **object detection**, **self-driving**, **medical image analysis**, or **language translation** involve many layers of abstraction — best handled by deep architectures.

5. Real-time or end-to-end learning is required
Deep learning models can often handle **end-to-end pipelines** (input to output without intermediate manual steps).

Example: Self-driving cars use DL to directly map camera input to steering control.

Deep Learning (DL) (Cont.)

Avoid Deep Learning If:

You have small datasets (DL will likely overfit).

You need transparent or interpretable models (ML models like decision trees or logistic regression are better).

You lack computational resources (DL requires GPUs/TPUs and more time).

The problem is simple or low-dimensional.

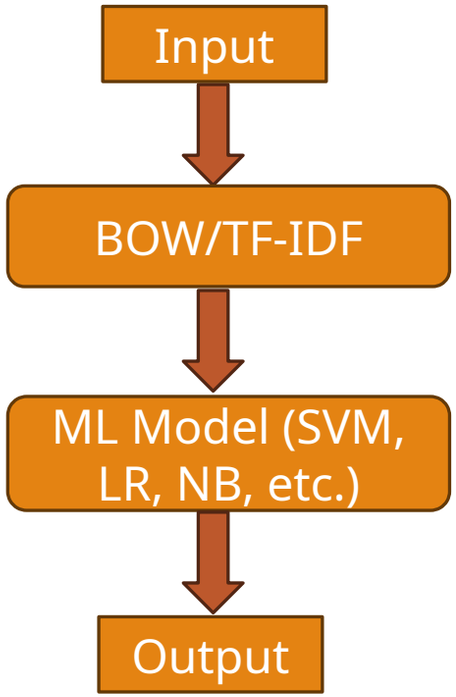
Deep Learning (DL) (Cont.)

Situation	Use ML	Use DL
Small tabular data	✅	❌
Image/video/audio data	❌	✅
Text understanding (complex NLP)	❌	✅
Few features, easy patterns	✅	❌
Large, complex data	❌	✅
Need explainability	✅	❌
Limited computing power	✅	❌

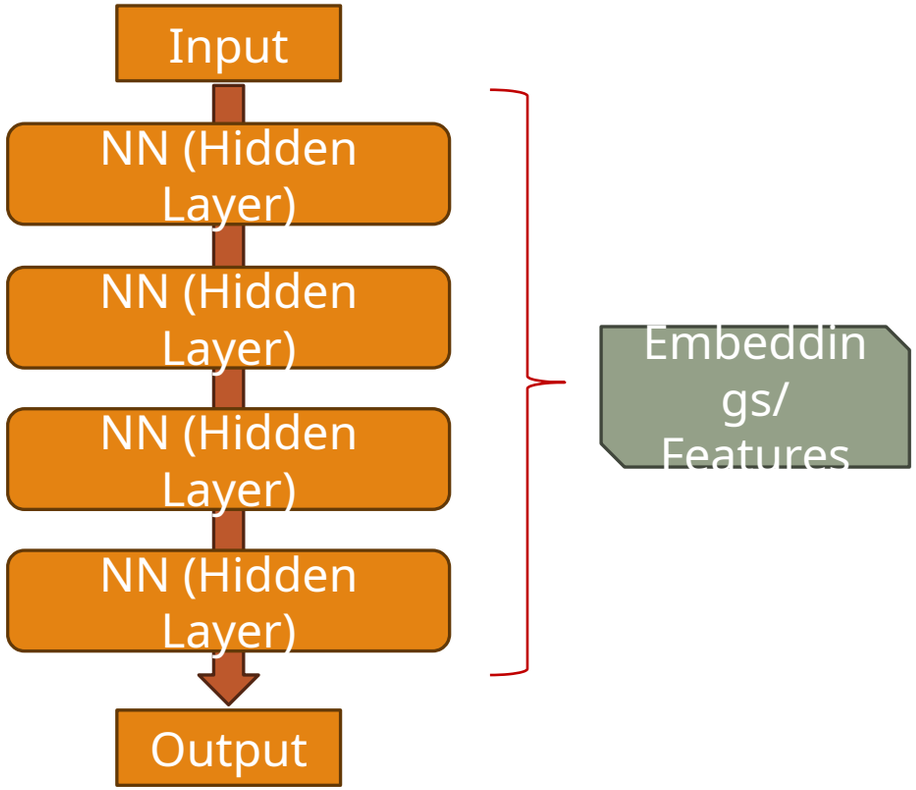
Workflow: NLP Powered by ML/DL

Input: "The movie was amazing!"
→ Preprocess (tokenize, lowercase)
→ Convert to vector (e.g., embedding)
→ DL model (LSTM or BERT)
Output: "Positive Sentiment"

Before/After Deep learning

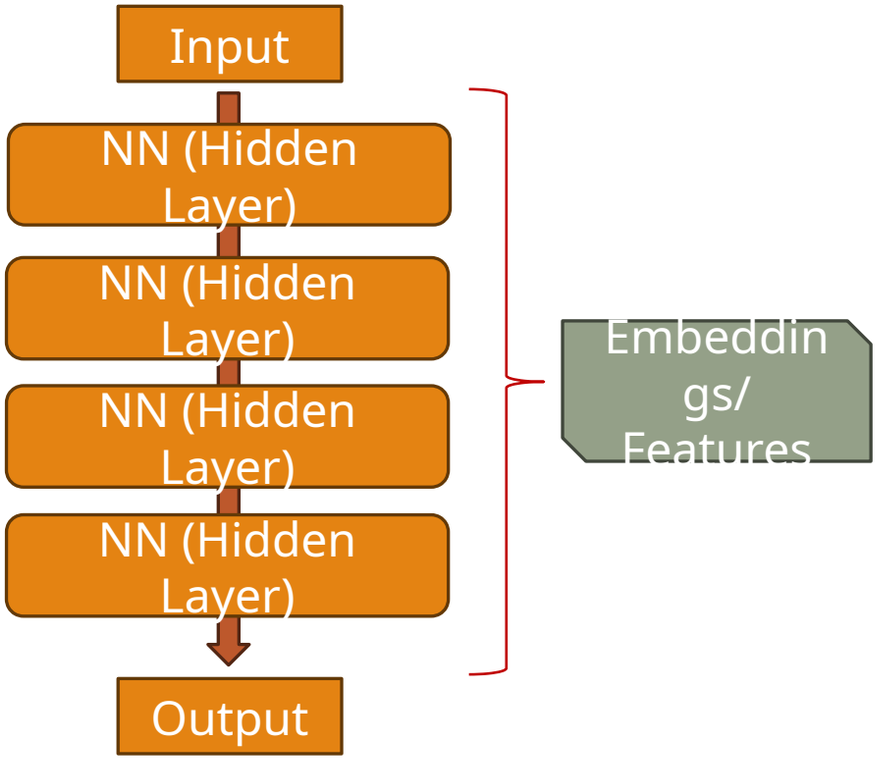


Before Deep Learning

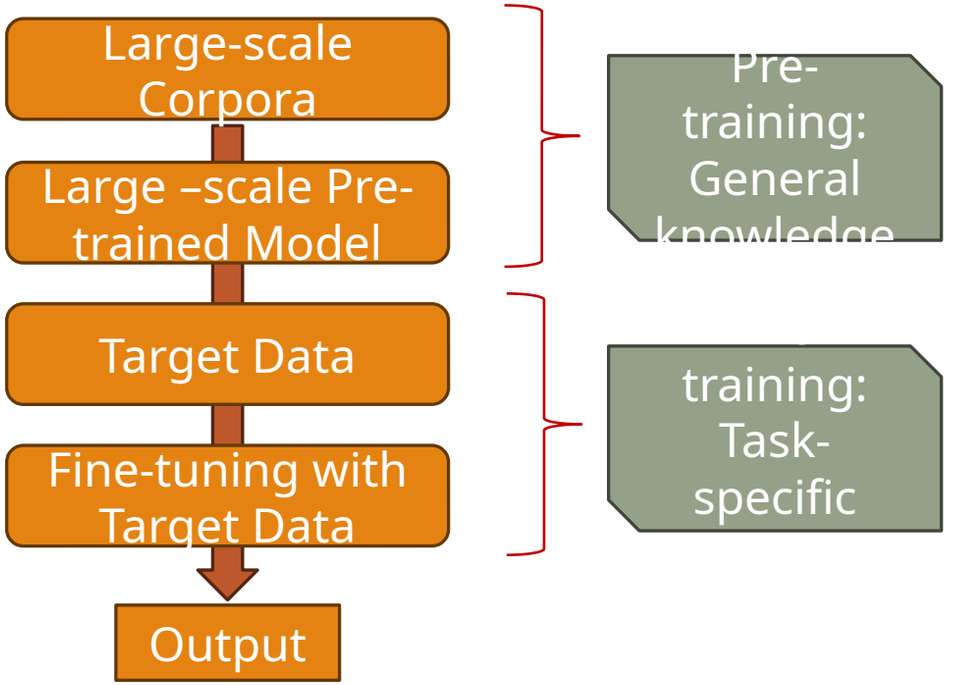


After Deep Learning

Before/After BERT or LLM



Deep learning (CNN, RNN, LSTM, etc.)



Deep learning (BERT/LLM-era)

Thank You

Machine Learning (ML)

- Machine Learning (ML) is a subset of artificial intelligence (AI) that enables systems to **learn from data and make predictions or decisions** without being explicitly programmed.
- It is about using algorithms that can **identify patterns in data and improve over time** with more data (experience).
- Example: A spam filter that learns to detect spam emails by analyzing features (contents, words, sender info, URLs, etc.) in previous spam and non-spam emails.

Basic Terminology in ML

Term	Description
Model	The mathematical representation of a real-world process learned from data.
Training	The process of feeding data to an algorithm to help it learn patterns.
Testing	Evaluating the trained model on unseen data.
Features	Input variables used to make predictions (e.g., height, age, keywords).
Labels	The desired output or target (e.g., spam or not spam).

Basic Terminology in ML (Cont.)

Term	Description
Algorithm	A method or process used to build the model from data.
Overfitting	When a model performs well on training data but poorly on new data.
Underfitting	When a model is too simple to learn the data patterns.
Supervised Learning	Learning from labeled data (input + output known).
Unsupervised Learning	Learning patterns from unlabeled data (no output).
Reinforcement Learning	Learning through rewards and penalties by interacting with an environment.

Deep Learning (DL)

- Deep Learning (DL) is a specialized subfield of machine learning that uses artificial neural networks, especially deep (multi-layered) networks.
- It is particularly powerful for large-scale and unstructured data like:

- Images
- Audio
- Text
- Video

Deep Learning (DL) (Cont.)

Use Deep Learning When:

1. You have a large dataset
Deep learning models need **a lot of data** to perform well. If you have **millions of samples**, deep learning can outperform classical methods.

Example: ImageNet with 14 million labeled images is ideal for training deep neural networks like CNNs.

Deep Learning (DL) (Cont.)

2. Your data is unstructured
•Images
•Audio/Speech
•Videos
•Natural Language (Text)

Examples:
•Image classification (cats vs. dogs) → Use CNN
•Speech recognition → Use RNN or Transformers
•Sentiment analysis or machine translation → Use LSTM or Transformers (e.g., BERT, GPT)

Deep Learning (DL) (Cont.)

3. You need automatic feature extraction
In traditional ML, **feature engineering** is done manually. Deep learning learns features directly from raw data using multiple layers.

Example: In facial recognition, CNNs automatically learn edges, shapes, and facial structures.

4. The problem is too complex for rule-based or simple ML models

Tasks like **object detection**, **self-driving**, **medical image analysis**, or **language translation** involve many layers of abstraction — best handled by deep architectures.

5. Real-time or end-to-end learning is required
Deep learning models can often handle **end-to-end pipelines** (input to output without intermediate manual steps).

Example: Self-driving cars use DL to directly map camera input to steering control.

Deep Learning (DL) (Cont.)

Avoid Deep Learning If:

You have small datasets (DL will likely overfit).

You need transparent or interpretable models (ML models like decision trees or logistic regression are better).

You lack computational resources (DL requires GPUs/TPUs and more time).

The problem is simple or low-dimensional.

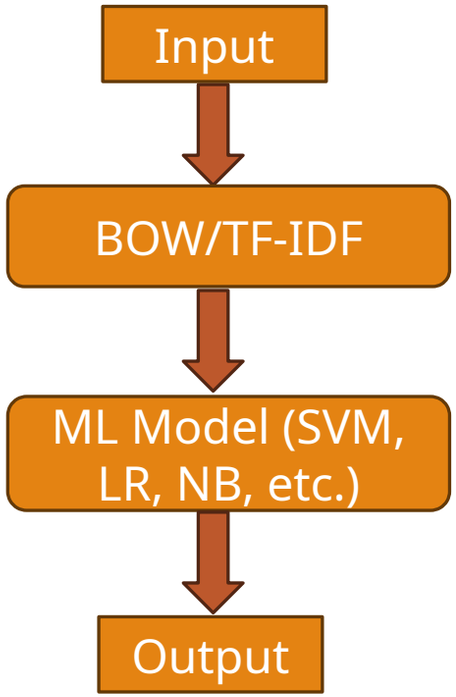
Deep Learning (DL) (Cont.)

Situation	Use ML	Use DL
Small tabular data	✅	❌
Image/video/audio data	❌	✅
Text understanding (complex NLP)	❌	✅
Few features, easy patterns	✅	❌
Large, complex data	❌	✅
Need explainability	✅	❌
Limited computing power	✅	❌

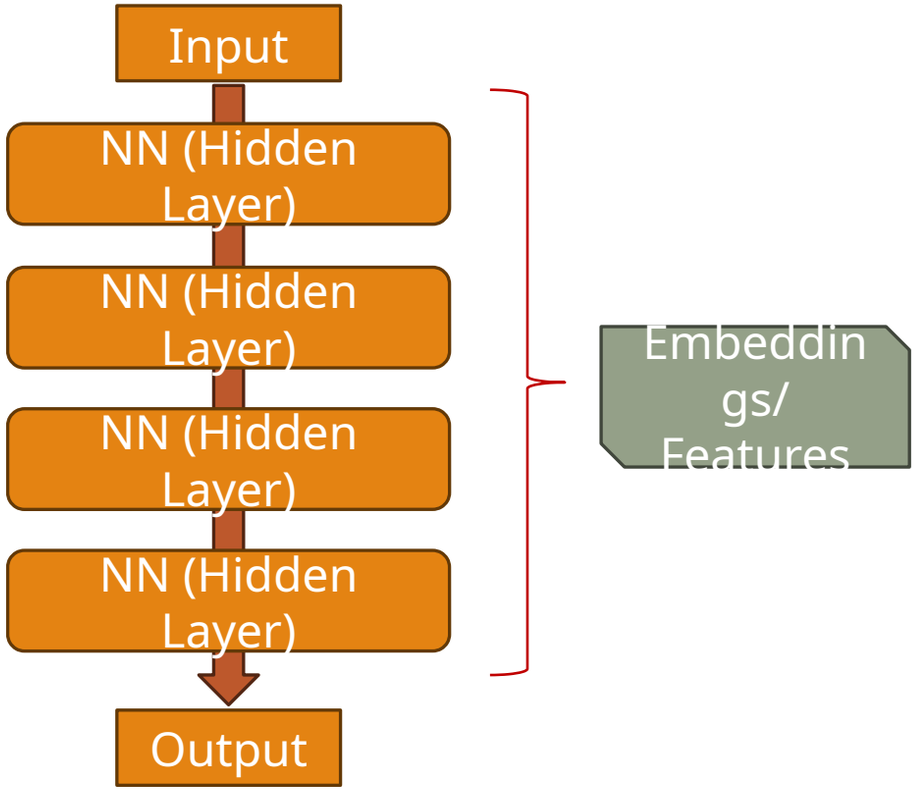
Workflow: NLP Powered by ML/DL

Input: "The movie was amazing!"
→ Preprocess (tokenize, lowercase)
→ Convert to vector (e.g., embedding)
→ DL model (LSTM or BERT)
Output: "Positive Sentiment"

Before/After Deep learning

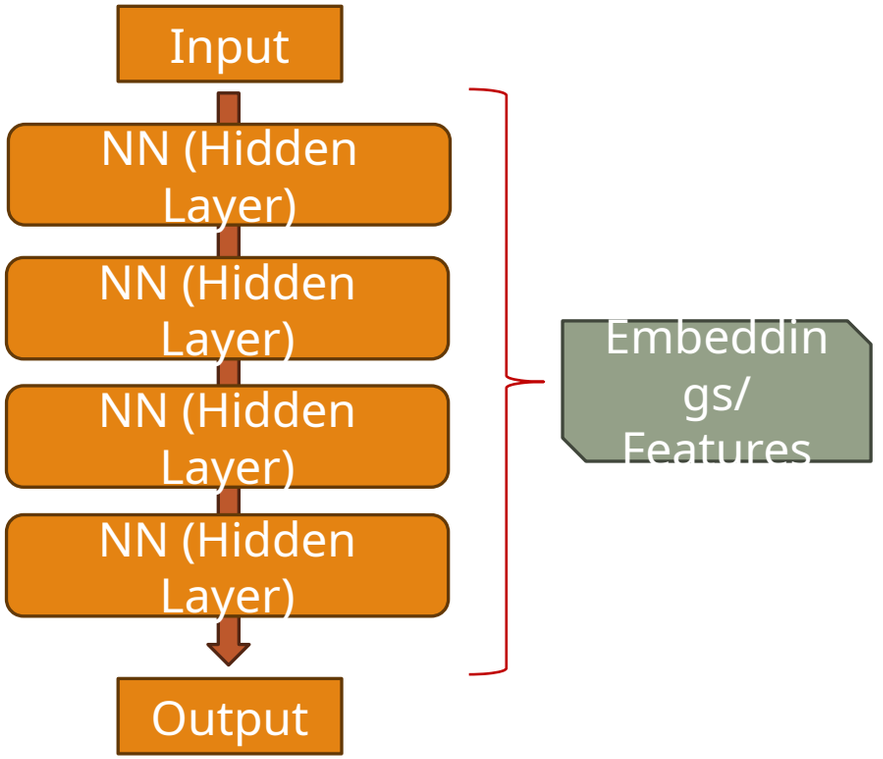


Before Deep Learning

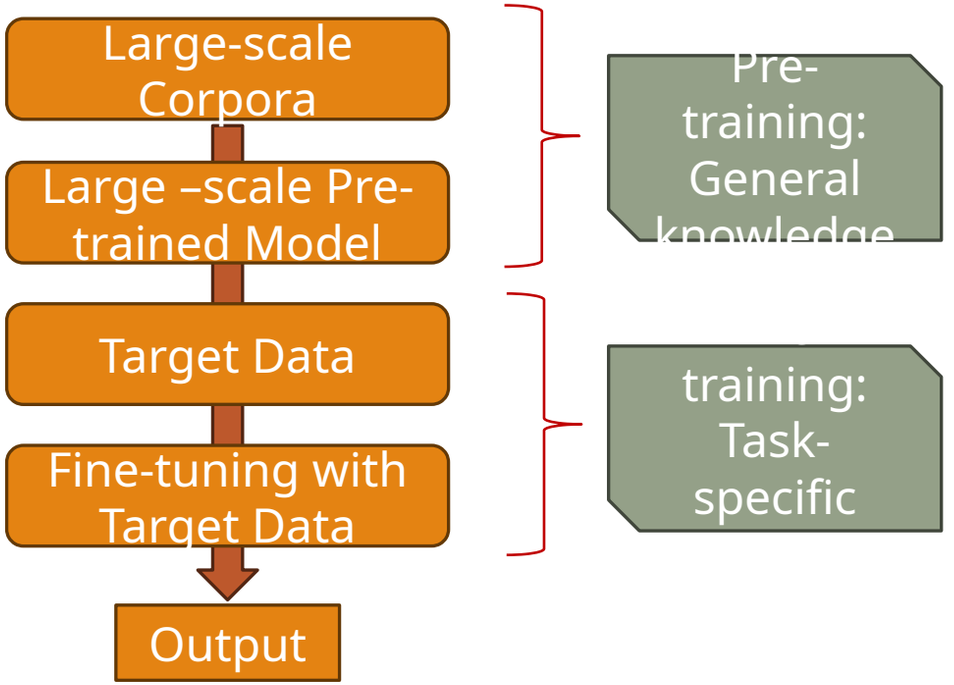


After Deep Learning

Before/After BERT or LLM



Deep learning (CNN, RNN, LSTM, etc.)



Deep learning (BERT/LLM-era)

General ML Algorithm

Step 1: Problem Definition

Understand the type of problem:

- **Classification** (e.g., spam detection)
- **Regression** (e.g., price prediction)
- **Clustering** (e.g., customer segmentation)

Step 2: Data Collection

Gather data from databases, APIs, sensors, files, etc.

Ensure data has features (X) and labels (y) (for supervised learning).

General ML Algorithm (Cont.)

Step 3: Data Preprocessing

Clean the data:

Handle missing values, remove duplicates.

Feature encoding:

Convert text/categorical data → numerical (e.g., one-hot encoding).

Scaling/Normalization:

StandardScaler/MinMaxScaler for numerical features.

Split the data:

Usually, 60–70% training, 20% validation, 10–20% test.

General ML Algorithm (Cont.)

Step 4: Model Selection

Problem	Examples
Classification	Logistic Regression, SVM, Decision Tree, Random Forest
Regression	Linear Regression, Ridge, Lasso
Clustering	K-Means, DBSCAN
NLP	Naive Bayes, RNN, BERT

General ML Algorithm (Cont.)

Step 5: Model Training

Fit the model to your training data

Step 6: Model Validation

Use validation data to tune parameters and prevent overfitting:

- Cross-validation (e.g., K-Fold)
- Hyperparameter tuning (Grid Search, Random Search)

General ML Algorithm (Cont.)

Step 7: Model Evaluation

Use test data to measure generalization performance.

Metric	Use
Accuracy	Classification
Precision, Recall, F1	Imbalanced classification
RMSE, MAE	Regression

General ML Algorithm (Cont.)

Step 8: Deployment (Optional)

Save the trained model (e.g., using Joblib, Pickle, or PyTorch/TensorFlow SavedModel)

Use it in web apps, mobile apps, or APIs.

Step 9: Monitoring & Updating

Monitor model performance on real data and retrain with new data if performance degrades.

Example: Iris Flower Classification Using Logistic Regression

Dataset: Iris (built into sklearn or UCI ML repository)

Dataset Characteristics: Tabular

Subject Area: Biology

Feature Type: Real

of Instances: 150

of Features: 4 (sepal length, sepal width, petal length, petal width)

Associated Tasks: Classification (3-class problem: Iris Setosa, Iris Versicolour, or Iris Virginica)

Example: Iris Flower Classification Using Logistic Regression (Cont.)

Model: Logistic Regression (LR)

Steps:

- Load and split data
- Train the model
- Validate using cross-validation
- Test on separate test data
- Report accuracy or F1 score

Learning Paradigm

1. Supervised Learning

The model learns from **labeled data —** i.e., input features with correct output.

Goal: **Predict the label** of new, unseen data.

Examples:

Classification (output is category):

- Spam vs Not Spam
- Cat vs Dog

Regression (output is number):

- House price prediction
- Temperature forecasting

Learning Paradigm (Cont.)

Algorithms:

Logistic Regression

Decision Trees

Support Vector Machines (SVM)

Neural Networks

Learning Paradigm (Cont.)

2. Unsupervised Learning

The model learns from **unlabeled data** — no output labels.

Goal: **Find hidden structure** (e.g., clusters, patterns).

Examples:

Clustering: Grouping similar items

- Market segmentation
- Document clustering

Dimensionality Reduction:

- PCA (reduce features)
- t-SNE (visualization)

Learning Paradigm (Cont.)

Algorithms:

K-Means

Hierarchical Clustering

DBSCAN

PCA

Learning Paradigm (Cont.)

3. Reinforcement Learning (RL)

A model (called an **agent**) learns by interacting with an **environment**.

It receives **rewards** or **penalties** based on actions.

Goal: **Maximize total reward** over time by learning a strategy (policy).

Learning Paradigm (Cont.)

Examples:

Game AI (Chess, Go, Atari)

Robotics (walking, navigation)

Self-driving cars

Resource management

Learning Paradigm (Cont.)

Term	Description
Agent	Learner/decision maker
Environment	Where agent operates
State	Current situation
Action	What agent can do
Reward	Feedback (+/-) for action
Policy	Strategy the agent learns

Learning Paradigm (Cont.)

Algorithms:

Q-Learning

Deep Q-Network (DQN)

Proximal Policy Optimization (PPO)

Learning Paradigm (Cont.)

Summary Comparison Table

Feature	Supervised	Unsupervised	Reinforcement
Data Type	Labeled	Unlabeled	Environment + rewards
Goal	Predict labels	Discover structure	Learn actions for max reward
Output	Class/number	Groups/patterns	Policy (action strategy)
Training Style	Input → Output	Input only	Trial & error
Example	Email classifier	Customer clusters	Game-playing bot

Pre-training, Fine-tuning, and Transfer Learning

1. Pre-training

Pre-training is the **initial learning phase** where a model learns from a **large, generic dataset** without focusing on any specific task.

The model learns **general patterns** in language or images.

It builds a **base of knowledge** that can be reused.

Pre-training, Fine-tuning, and Transfer Learning (Cont.)

📚 NLP Example:

BERT is pre-trained on massive text corpora (like Wikipedia) using:

- Masked language modeling (predict missing words)
- Next sentence prediction

📸 Vision Example:

ResNet is pre-trained on ImageNet to recognize 1000 general object classes.

Pre-training, Fine-tuning, and Transfer Learning (Cont.)

2. Fine-tuning

Fine-tuning is the **second stage**, where the **pre-trained model** is adapted to a **specific task or dataset**.

Starts with a pre-trained model

Retrains the model (usually partially) on **smaller, task-specific data**

Pre-training, Fine-tuning, and Transfer Learning (Cont.)

📚 **NLP Example:**

Fine-tune BERT for **sentiment analysis** on a product review dataset.

📸 **Vision Example:**

Fine-tune ResNet to classify **types of medical images** (e.g., X-rays)

Pre-training, Fine-tuning, and Transfer Learning (Cont.)

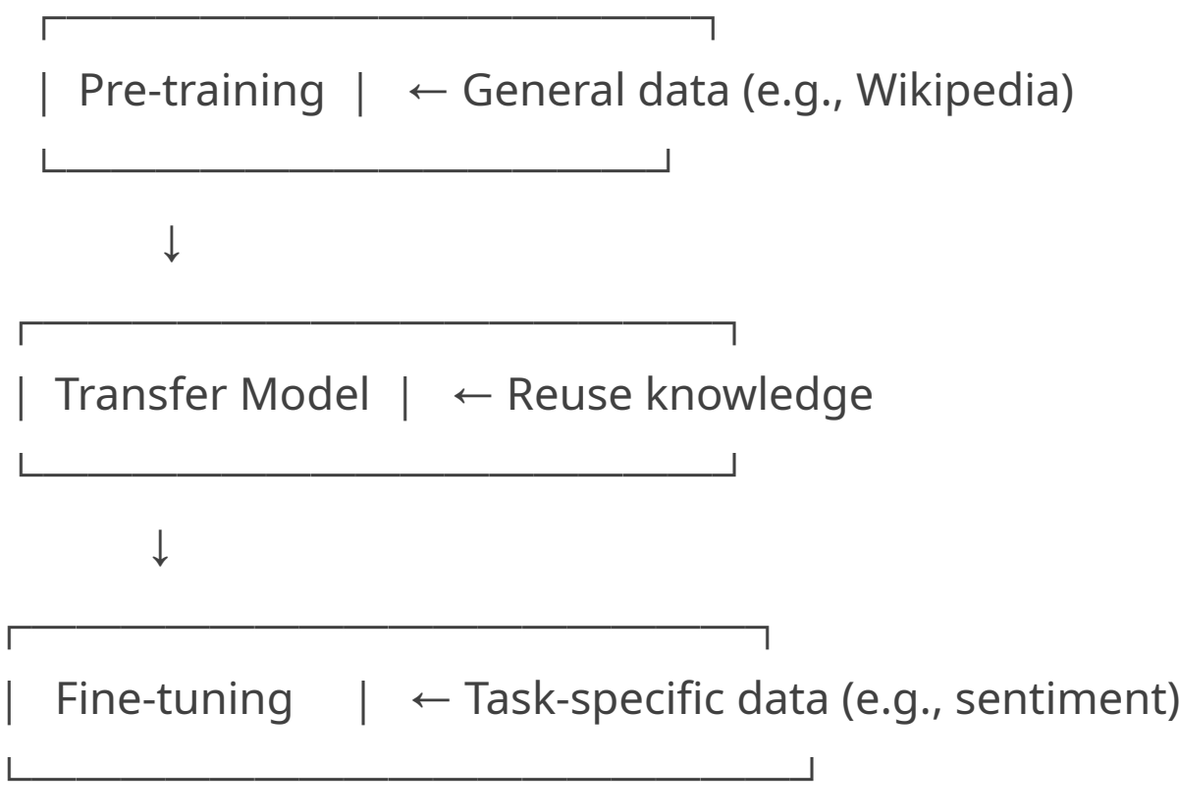
3. Transfer Learning

Transfer Learning is the **overall concept** of reusing a model trained on one task (source) for another related task (target).

- It **includes both pre-training and fine-tuning**.
- You **transfer knowledge** from a large, general model to a smaller, specific problem.

In practice, transfer learning = **Pre-trained model + Fine-tuning**

Pre-training, Fine-tuning, and Transfer Learning (Cont.)



Pre-training, Fine-tuning, and Transfer Learning (Cont.)

Phase	Dataset	Task
Pre-training	BooksCorpus + Wikipedia	Learn general language structure
Fine-tuning	Movie reviews	Sentiment classification
Transfer learning	Using BERT’s weights on any NLP task	Better performance

Pre-training, Fine-tuning, and Transfer Learning (Cont.)

Concept	What it Does	Data Needed	Cost	Benefit
Pre-training	Learn general knowledge	Very large	Expensive	General reusable model
Fine-tuning	Adapt to specific task	Small, labeled	Low	Specialization
Transfer Learning	Transfer knowledge to new task	Target-specific	Low	Improves performance

Practical Applications of ML

1. Natural Language Processing (NLP)

Spam detection (e.g., Gmail filters)

Sentiment analysis (e.g., product reviews, social media)

Chatbots and virtual assistants (e.g., Siri, ChatGPT)

Machine translation (e.g., Google Translate)

Text summarization and classification

Practical Applications of ML (Cont.)

2. Computer Vision

Facial recognition (e.g., smartphone unlock, surveillance)

Medical image analysis (e.g., detecting cancer in scans)

Self-driving cars (e.g., lane detection, object tracking)

Practical Applications of ML (Cont.)

3. Healthcare

Disease prediction (e.g., diabetes, cancer, heart disease)

Medical diagnostics using X-rays, MRIs, ECGs

4. Finance and Business

Fraud detection in banking

Chatbots for customer service

Product review analysis

Practical Applications of ML (Cont.)

5. E-commerce and Retail

Recommendation systems (e.g., Amazon, Netflix)

Inventory forecasting

Visual search (e.g., upload image to find similar products)

6. Transportation and Logistics

Route optimization (e.g., Uber, FedEx)

Self-driving technology (e.g., Tesla Autopilot)

Practical Applications of ML (Cont.)

7. Gaming and Entertainment

Game AI (e.g., NPC behavior, reinforcement learning agents)

Music recommendation (e.g., Spotify)

8. Agriculture

Crop disease prediction from leaf images

Yield estimation

Smart irrigation systems

Drone-based crop monitoring

Practical Applications of ML (Cont.)

9. Environment and Climate

Weather forecasting

Earthquake and flood prediction

10. Government and Public Sector

Crime prediction and prevention

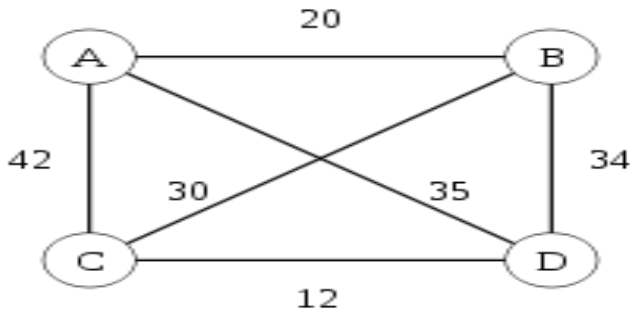
Smart city planning

Citizen sentiment analysis

Thank You

Travelling Salesman Problem

- **Given a list of cities and the distances between each pair of cities.**
- **What is the shortest possible routes that visits each city exactly once and returns to the original city?**
- **Exhaustive search is not feasible.**
- **It is an NP-hard problem in combinatorial optimization.**



Techniques

- Brute-force approach
  - Check every possible route and return the shortest route.
  - Time complexity – O(n!)
  - 10! is 3,628,800
  - 20! is 2432902008176640000
  - 30! is ??

Techniques

- Dynamic Programming
  - Keep optimal sub-routes of certain length with their end-point.
  - Then add a point to the routes and figure out optimal sub-routes that are now one point longer then in the previous step.
  - Keep doing this until we get to the full route.
  - Time complexity –$O(n^2.2^n)$
  - $10^2 2^{10} = 102{,}400$
  - $20^2 2^{20} = 419430400$
  - $30^2 2^{30} = 966367641600$
  - $40^2 2^{40} = ??$

Heuristic Approach

- A heuristic is a technique designed for solving a problem **more quickly** when classic methods are too slow
- Or, for finding an approximate solution when **classic methods fail to find any exact solution**.

- Results about NP-hardness in theoretical computer science make heuristics the only viable option for a variety of complex optimization problems that need to be routinely solved in real-world applications.

Heuristic Approach (cont.)

- The trade-off criteria for deciding whether to use a heuristic for solving a given problem include the following:
- ***Optimality:*** When several solutions exist for a given problem, does the heuristic guarantee that the best solution will be found? Is it actually necessary to find the best solution?
- ***Completeness:*** When several solutions exist for a given problem, can the heuristic find them all? Do we actually need all solutions? Many heuristics are only meant to find one solution.

Heuristic Approach (cont.)

- ***Accuracy and precision****:* Can the heuristic provide a confidence interval for the purported solution? Is the error bar on the solution unreasonably large?
- ***Execution time*****:** Is this the best-known heuristic for solving this type of problem? Some heuristics converge faster than others. Some heuristics are only marginally quicker than classic methods.

- **Bio-inspired heuristic approach:**
  - Genetic Algorithm (GA)
  - Particle Swarm Optimization (PSO)
  - Ant Colony Optimization (ACO)
  - Artificial Bee Colony Optimization (ABCO)
  - Cuckoo Search (CS)
  - ……

Introduction To Genetic Algorithm (GA)_

What Are Genetic Algorithms (GA)?

GA is a *search* and *optimization* technique based on Darwin's Principle of *Natural Selection*.

Darwin's Principle Of Natural Selection

- IF there are organisms that reproduce, and
- IF offsprings inherit traits (genetically determined characteristics) from their progenitors (ancestors or parents), and
- IF there is variability of traits, and
- IF the environment cannot support all members of a growing population,
- THEN those members of the population with less-adaptive traits (determined by the environment) will die out, and
- THEN those members with more-adaptive traits (determined by the environment) will thrive.

Basic Idea Of Principle Of Natural Selection

“Select The Best, Discard The Rest”

An Example of Natural Selection

- **Giraffes have long necks.**

Giraffes with slightly longer necks could feed on leaves of higher branches when all lower ones had been eaten off.

→ They had a better chance of survival.

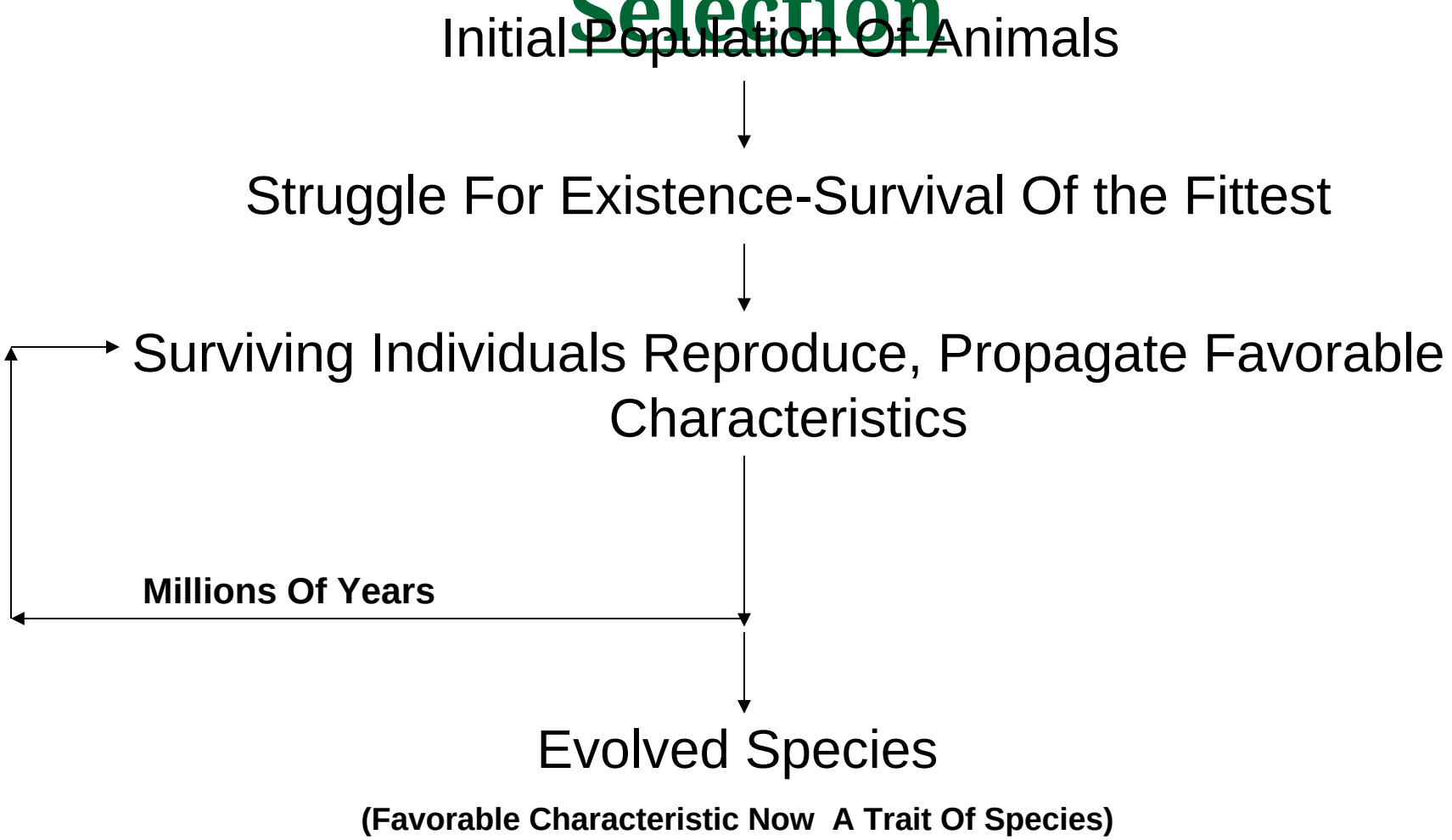
→ Favorable behaviors propagated through generations of giraffes.

→ Now, evolved species has long necks.

NOTE: Longer necks may have been a deviant characteristic (mutation) initially but since it was favorable, was propagated over generations.

So, some mutations are beneficial.

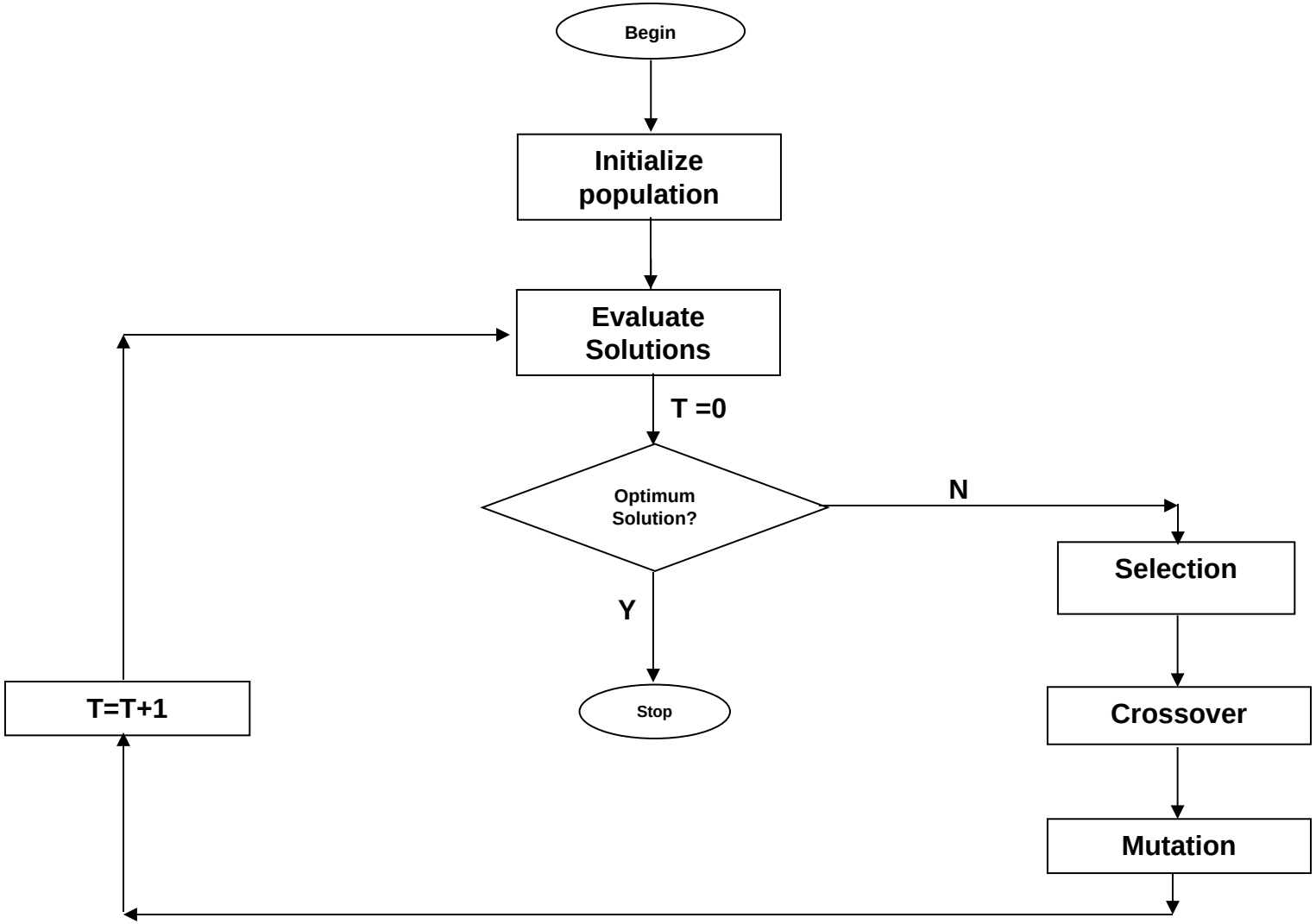
Evolution Through Natural Selection



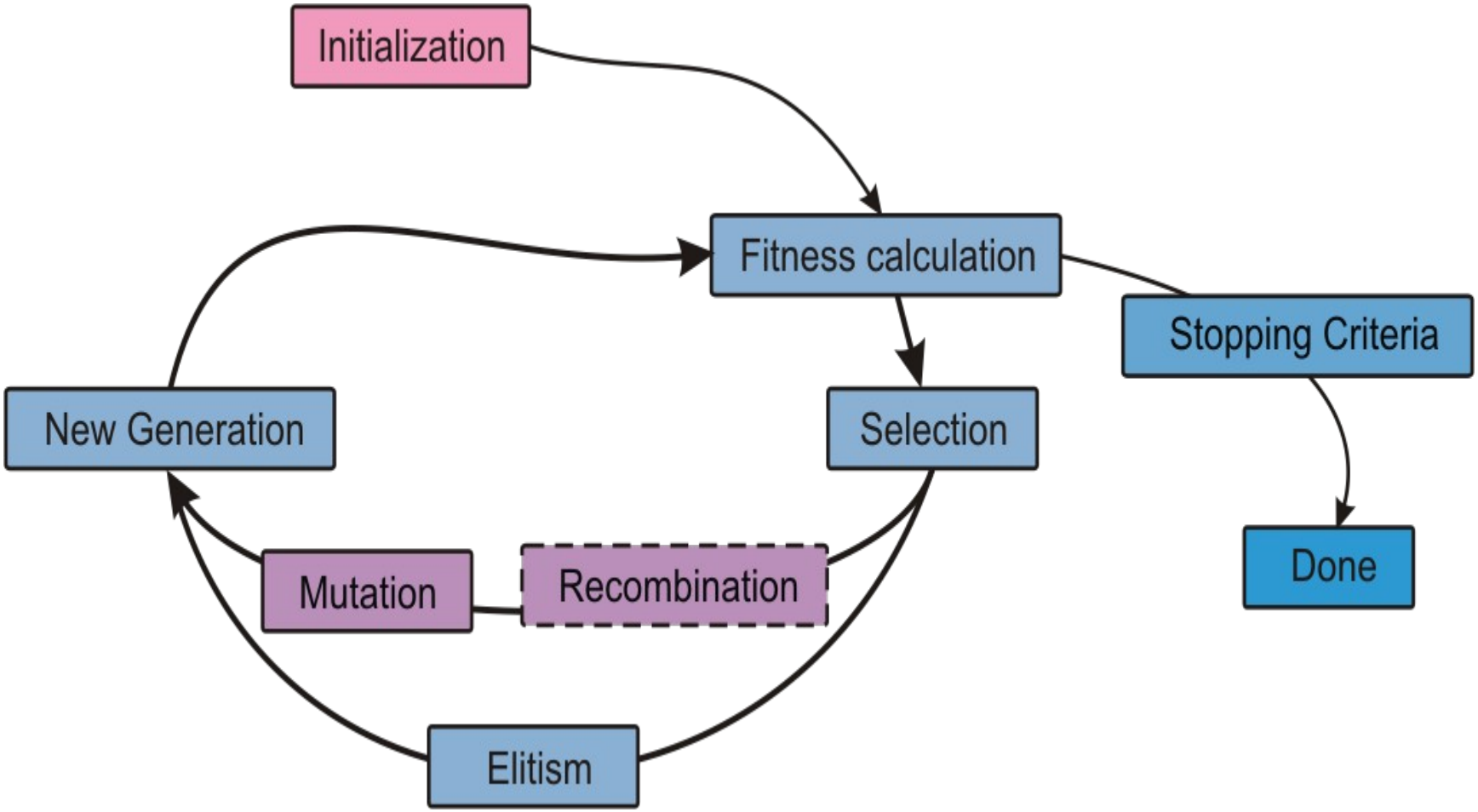


GA Implement Optimization Strategies By Simulating Evolution Of Species Through Natural Selection

Working Mechanism Of GAs



The Cycle of Evolution



Simple Genetic Algorithm

```
Simple_Genetic_Algorithm()
        {
        Initialize the Population;
        Calculate Fitness Function;

        While(Fitness Value != Optimal Value)
                {
                Selection;//Natural Selection, Survival Of
   the Fittest
                Crossover;//Reproduction, Propagate favorable
   characteristics

                Mutation;//Mutation
                }
     Calculate Fitness Function;
        }
```

Nature to Computer Mapping

Nature	Computer
Population	Set of solutions.
Individual	Solution to a problem.
Fitness	Quality of a solution.
Chromosome	Encoding for a solution.
Gene	Part of the encoding of a solution.
Reproduction	Crossover

GA Operators and Parameters

Encoding

*The process of representing the solution in the form of a **string** that conveys the necessary information.*

- Just as in a chromosome, each gene controls a particular characteristic of the individual, similarly, each bit in the string represents a characteristic of the solution.

Encoding Methods

- **Binary Encoding –** Most common method of encoding. Chromosomes are strings of 1s and 0s and each position in the chromosome represents a particular characteristic of the problem.

Chromosome A	1011001011001100101
Chromosome B	1111111000000011111

Encoding Methods (contd.)

- **Permutation Encoding –** Useful in ordering problems such as the Traveling Salesman Problem (TSP).
- Example. In TSP, every chromosome is a string of numbers, each of which represents a city to be visited.

Chromosome A	1 5 3 2 6 4 7 9 8
Chromosome B	8 5 6 7 2 3 1 4 9

Encoding Methods (contd.)

- **Value Encoding** – Used in problems where complicated values, such as real numbers, are used and where binary encoding would not suffice.

  Good for some problems, but *often necessary to develop some specific crossover and mutation techniques for these chromosomes.*

Chromosome A	1.235 5.323 0.454 2.321 2.454
Chromosome B	(left), (back), (left), (right), (forward)

Fitness Function

A fitness function quantifies the optimality of a solution (chromosome) so that particular solution may be ranked against all the other solutions.

- A fitness value is assigned to each solution depending on how close it actually is to solving the problem.
- An ideal fitness function should correlate closely with the problem's goal, accurately measuring how well a solution solves the objective, and be quickly computable.
- Example. In TSP, f(x) is sum of distances between the cities in solution. The lesser the value, the fitter the solution is.

Selection

The process that determines which solutions are to be preserved and allowed to reproduce and which ones deserve to die out.

- The primary objective is to emphasize the good solutions and eliminate the bad solutions in a population, while keeping the population size constant.
- "Selects The Best, Discards The Rest".

Selection

- Identify the good solutions in a population.
- Make multiple solutions from the good solutions.
- Eliminate bad solutions from the population so that multiple copies of good solutions can be placed in the population.

Roulette Wheel Selection

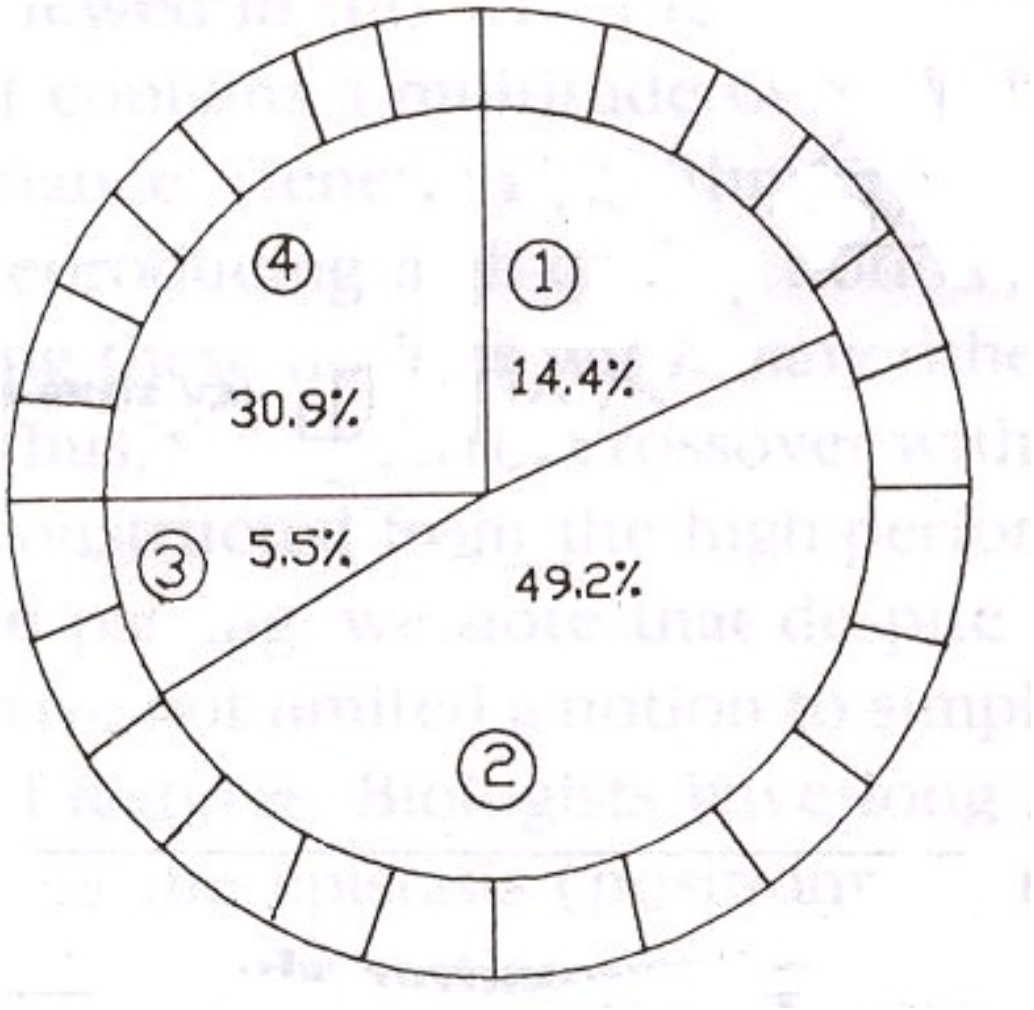
- Each current string in the population has a slot assigned to it which is in proportion to its fitness.
- We spin the weighted *roulette wheel* thus defined *n* times (where *n* is the total number of solutions).
- Each time Roulette Wheel stops, the string corresponding to that slot is created.

Strings that are fitter are assigned a larger slot and hence have a better chance of appearing in the new population.

Example Of Roulette Wheel Selection

No.	String	Fitness	% Of Total
1	01101	169	14.4
2	11000	576	49.2
3	01000	64	5.5
4	10011	361	30.9
Total		1170	100.0

Roulette Wheel For Example



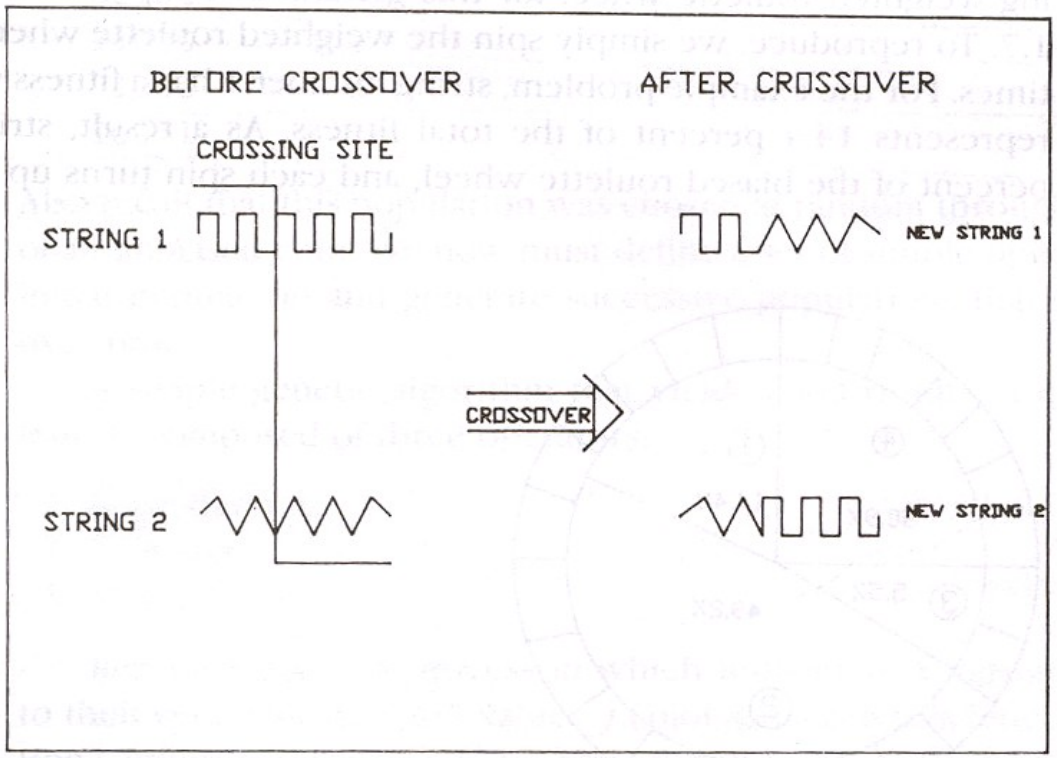
Crossover

It is the process in which two chromosomes (strings) combine their genetic material (bits) to produce a new offspring which possesses both their characteristics.

- *The method chosen depends on the Encoding Method.*

Crossover Methods

- **Single Point Crossover-** A random point is chosen on the individual chromosomes (strings) and the genetic material is exchanged at this point.



Crossover Methods (contd.)

- **Single Point Crossover**

Chromosome1	11011 | 00100110110
Chromosome 2	11011 | 11000011110
Offspring 1	11011 | 11000011110
Offspring 2	11011 | 00100110110

Crossover Methods (contd.)

- **Two-Point Crossover-** Two random points are chosen on the individual chromosomes (strings) and the genetic material is exchanged at these points.

Chromosome1	**11011 | 00100 | 110110**
Chromosome 2	**10101 | 11000 | 011110**
Offspring 1	**10101 | 00100 | 011110**
Offspring 2	**11011 | 11000 | 110110**

NOTE: These chromosomes are different from the last example.

Crossover Methods (contd.)

- **Uniform Crossover-** Each gene (bit) is selected randomly from one of the corresponding genes of the parent chromosomes.

Chromosome1	**11011 | 00100 | 110110**
Chromosome 2	**10101 | 11000 | 011110**
Offspring	**10111 | 00000 | 110110**

Crossover (contd.)

- Crossover between 2 good solutions **MAY NOT ALWAYS** yield a better or as good a solution.
- Since parents are good, probability of the child being good is high.
- If offspring is not good (poor solution), it will be removed in the next iteration during “Selection”.
- *Crossover Probability:* determines the number of chromosomes will take part in crossover.

Elitism

Elitism is a method which copies the best chromosome to the new offspring population before crossover and mutation.

- When creating a new population by crossover or mutation the best chromosome might be lost.
- Forces GAs to retain some number of the best individuals at each generation.
- Has been found that elitism significantly improves performance.

Mutation

It is the process by which a string is deliberately changed so as to maintain diversity in the population set.

We saw in the giraffes' example, that mutations could be beneficial.

Mutation Probability- determines how often the parts of a chromosome will be mutated.

Example Of Mutation

- For chromosomes using Binary Encoding, randomly selected bits are inverted.

Offspring	**11011 00100 110110**
Mutated Offspring	**11010 00100 100110**

NOTE: The number of bits to be inverted depends on the Mutation Probability.

When to Use a GA

- Alternate solutions are too slow or overly complicated
- Problem is similar to one that has already been successfully solved by using a GA
- Want to hybridize with an existing solution

Simple Example

- **Simple problem: max x^2 over {0,1,…,31}**
- GA approach:
  - Representation: binary code, e.g. 01101 ↔ 13
  - Population size: 4
  - 1-point xover, bitwise mutation
  - Roulette wheel selection
  - Random initialization
- We show one generational cycle done by hand

Selection

String no.	Initial population	x Value	Fitness $f(x) = x^2$	$Prob_i$	Expected count	Actual count
1	0 1 1 0 1	13	169	0.14	0.58	1
2	1 1 0 0 0	24	576	0.49	1.97	2
3	0 1 0 0 0	8	64	0.06	0.22	0
4	1 0 0 1 1	19	361	0.31	1.23	1
Sum			1170	1.00	4.00	4
Average			293	0.25	1.00	1
Max			576	0.49	1.97	2

Crossover

String no.	Mating pool	Crossover point	Offspring after xover	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 | 1	4	0 1 1 0 0	12	144
2	1 1 0 0 | 0	4	1 1 0 0 1	25	625
2	1 1 | 0 0 0	2	1 1 0 1 1	27	729
4	1 0 | 0 1 1	2	1 0 0 0 0	16	256
Sum					1754
Average					439
Max					729

Mutation

String no.	Offspring after xover	Offspring after mutation	x Value	Fitness $f(x) = x^2$
1	0 1 1 0 0	1 1 1 0 0	26	676
2	1 1 0 0 1	1 1 0 0 1	25	625
2	1 1 0 1 1	1 1 0 1 1	27	729
4	1 0 0 0 0	1 0 1 0 0	18	324
Sum				2354
Average				588.5
Max				729

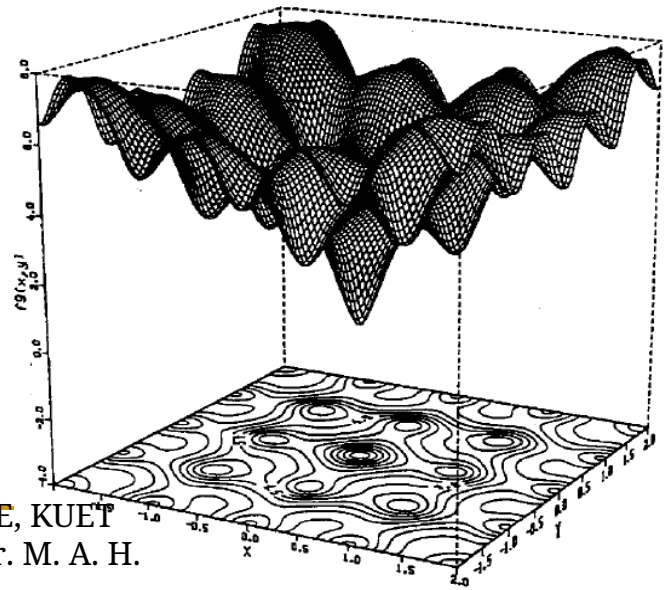
Hard Problems

- Solution of $f(x)=x^2$ is easy to find and not realistic for GA
- Many problems occur as real valued problems, e.g., continuous parameter optimization $f : \mathcal{R}^n \rightarrow \mathcal{R}$
- Illustration: Ackley's function (often used in EC)

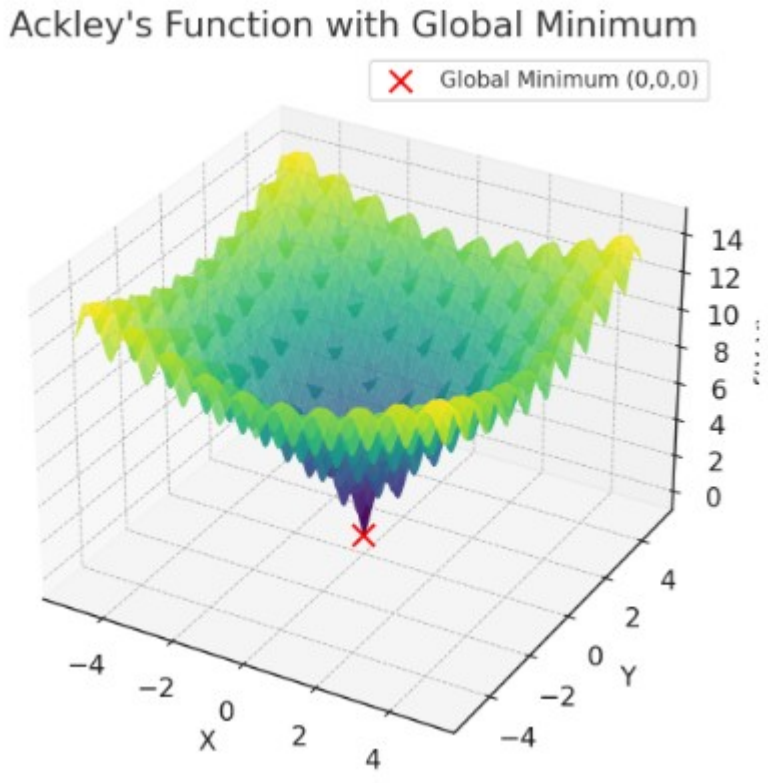
$$f(\overline{x}) = -c_1 \cdot exp\left(-c_2 \cdot \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}\right)$$

$$-exp\left(\frac{1}{n}\cdot\sum_{i=1}^{n} cos(c_3 \cdot x_i)\right) + c_1 + 1$$

$$c_1 = 20,\ c_2 = 0.2,\ c_3 = 2\pi$$



Hard Problems (Cont.)



Genetic Algorithms To Solve The Traveling Salesman Problem (TSP)

The Problem

The **Traveling Salesman Problem** is defined as:

'We are given a set of cities and a symmetric distance matrix that indicates the cost of travel from each city to every other city.

*The goal is to find **the shortest circular tour,** visiting every city exactly once, so as to **minimize the total travel cost**, which includes the cost of traveling from the last city back to the first city'.*

Encoding

- Represent every city with an integer.

- Consider 6 Indian cities –

  Mumbai, Nagpur , Calcutta, Delhi , Bangalore and Chennai and assign a number to each.

Mumbai → 1
Nagpur → 2
Calcutta → 3
Delhi → 4
Bangalore → 5
Chennai → 6

Encoding (contd.)

- Thus, a path would be represented as a sequence of integers from 1 to 6.
- The path [1 2 3 4 5 6] represents a path from Mumbai to Nagpur, Nagpur to Calcutta, Calcutta to Delhi, Delhi to Bangalore, Bangalore to Chennai, and finally from Chennai to Mumbai.
- This is an example of **Permutation Encoding** as the position of the elements determines the fitness of the solution.

Fitness Function

- The fitness function will be the total cost of the tour represented by each chromosome.
- This can be calculated as the sum of the distances traversed in each travel segment.

The Lesser The Sum, The Fitter The Solution Represented By That Chromosome.

Distance/Cost Matrix For TSP

	1	2	3	4	5	6
1	0	863	1987	1407	998	1369
2	863	0	1124	1012	1049	1083
3	1987	1124	0	1461	1881	1676
4	1407	1012	1461	0	2061	2095
5	998	1049	1881	2061	0	331
6	1369	1083	1676	2095	331	0

Cost matrix for six city example.

Distances in Kilometers

Fitness Function (contd.)

- So, for a chromosome [4 1 3 2 5 6], the total cost of travel or fitness will be calculated as shown below

- Fitness = *1407 + 1987 + 1124 + 1049 + 331+ 2095*

  = *7993 kms.*

- Since our objective is to Minimize the distance, the lesser the total distance, the fitter the solution.

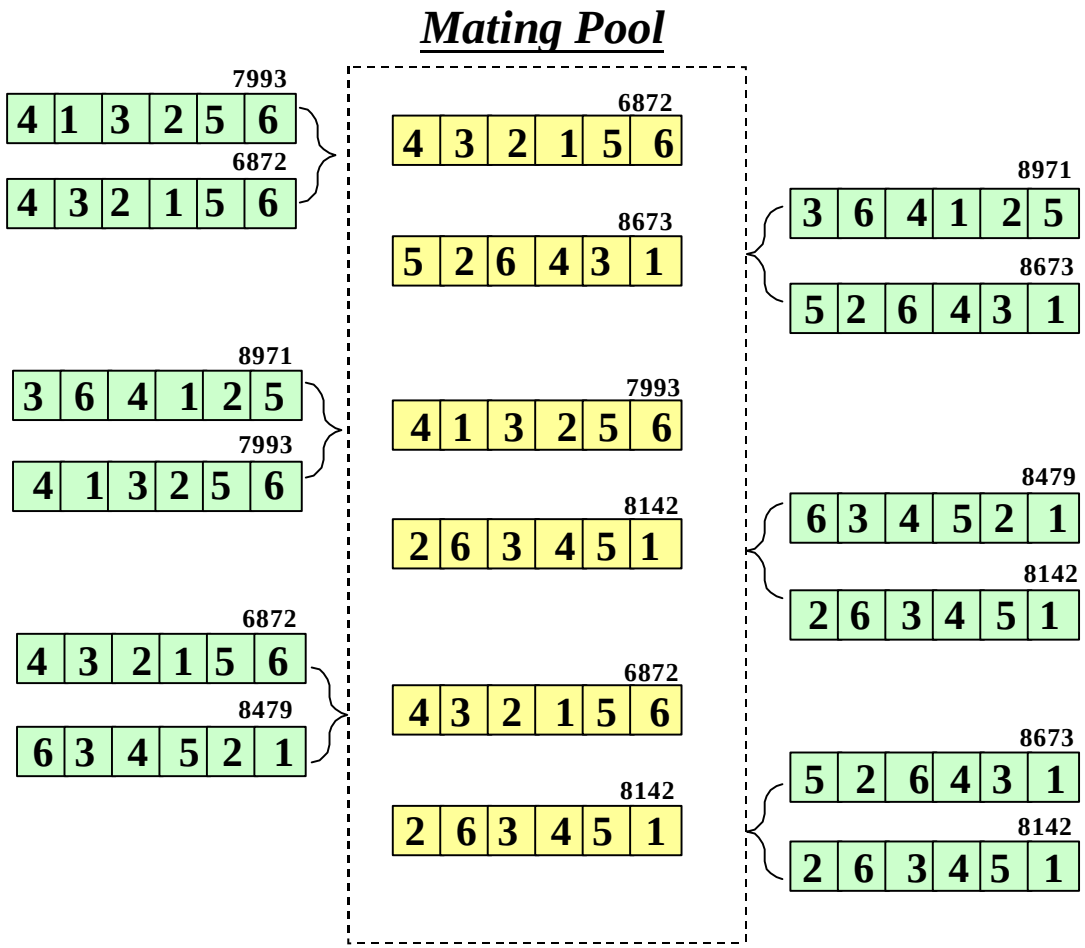
Selection Operator

May use ***Tournament Selection***.

As the name suggests *tournaments* are played between two solutions and the better solution is chosen and placed in the *mating pool*.

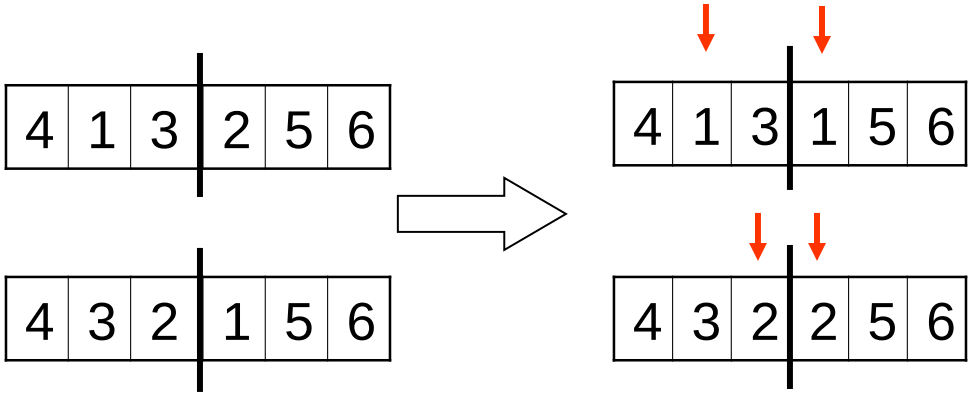
Two other solutions are picked again and another slot in the *mating pool* is filled up with the better solution.

Tournament Selection (contd.)



Why we cannot use single-point crossover:

- Single point crossover method randomly selects a crossover point in the string and swaps the substrings.
- This may produce some invalid offsprings as shown below.



Crossover Operator

- Used the *Enhanced Edge Recombination* operator (T.Starkweather et al., *'A Comparison of Genetic Sequencing Operators',* International Conference of GAs, 1991).

- This operator is different from other genetic sequencing operators in that it emphasizes *adjacency information* instead of the order or position of items in the sequence.

- The algorithm for the Edge-Recombination Operator involves constructing an *Edge Table* first.

Edge Table

The *Edge Table* is an *adjacency table* that lists links *into* and *out of* a city found in the two parent sequences.

If an item is already in the edge table and we are trying to insert it again, that element of a sequence must be a *common edge* and is represented by inverting its sign.

Finding The Edge Table

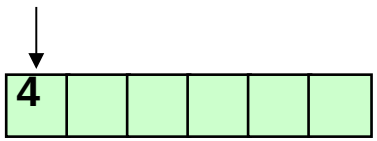
Parent 1	4	1	3	2	5	6
Parent 2	4	3	2	1	5	6

1	4	3	2	5
2	-3	5	1	
3	1	-2	4	
4	-6	1	3	
5	1	2	-6	
6	-5	-4		

Example Of Enhanced Edge Recombination Operator

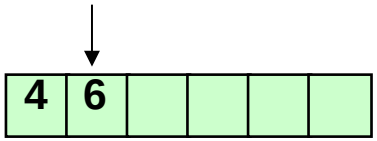
Step 1

1	4	3	2	5
2	-3	5	1	
3	1	-2	4	
4	-6	1	3	
5	3	2	-6	
6	-5	-4		



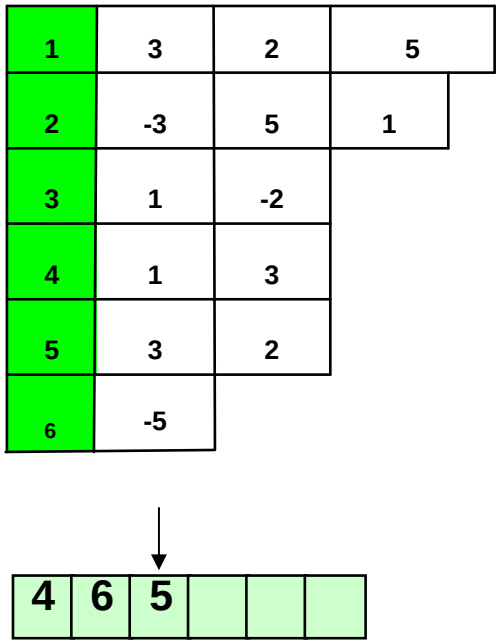
Step 2

1	3	2	5
2	-3	5	1
3	1	-2	
4	-6	1	3
5	3	2	-6
6	-5		

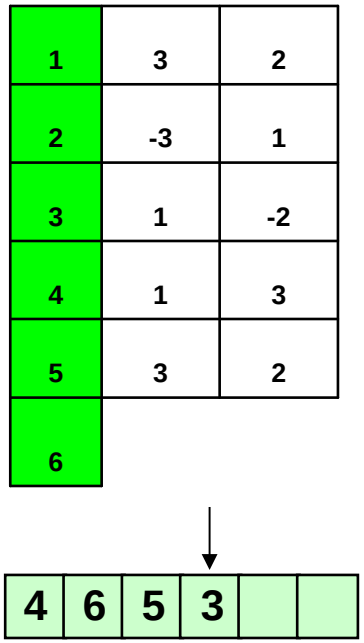


Example Of Enhanced Edge Recombination Operator (contd.)

Step 3

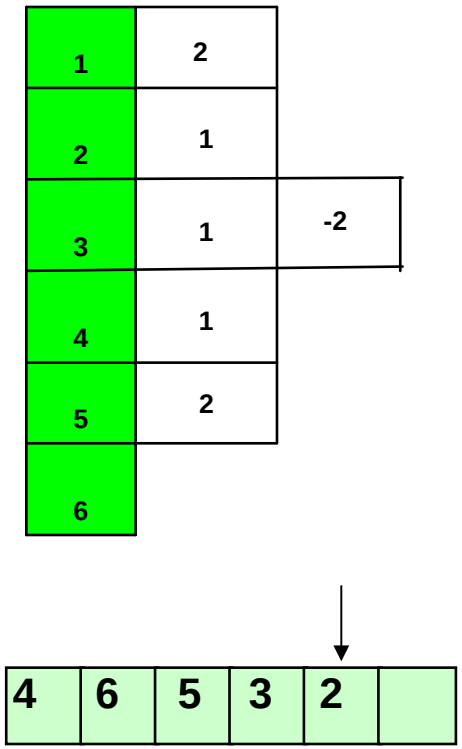


Step 4

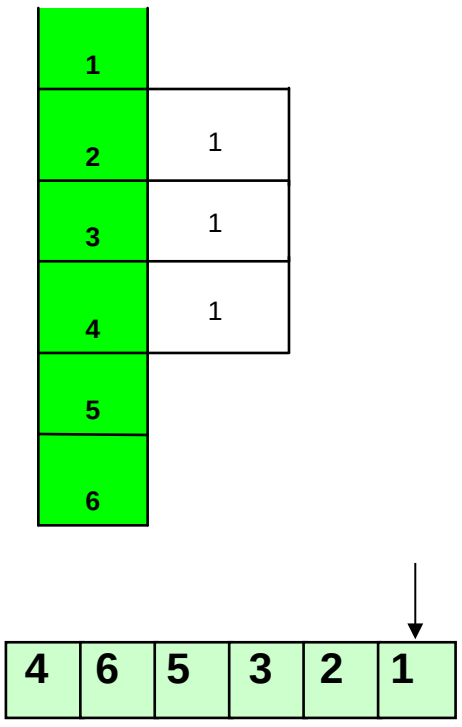


Example Of Enhanced Edge Recombination Operator (contd.)

Step 5

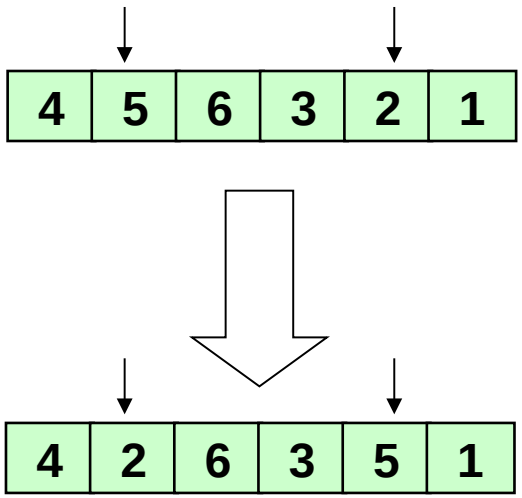


Step 6



Mutation Operator

- The mutation operator induces a change in the solution, so as to maintain diversity in the population and prevent *Premature Convergence*.
- In our project, we mutate the string by randomly selecting any two cities and interchanging their positions in the solution, thus giving rise to a new tour.



GA Applications

Domain	Application Types
Control	gas pipeline, pole balancing, missile evasion, pursuit
Design	semiconductor layout, aircraft design, keyboard configuration, communication networks
Scheduling	manufacturing, facility scheduling, resource allocation
Robotics	trajectory planning
Machine Learning	designing neural networks, improving classification algorithms, classifier systems
Signal Processing	filter design
Game Playing	poker, checkers, prisoner's dilemma
Combinatorial Optimization	set covering, travelling salesman, routing, bin packing, graph colouring and partitioning

Swarm Intelligence



What is a Swarm?

- A loosely structured collection of interacting agents
  - Agents:
    - Individuals that belong to a group
    - They contribute to and benefit from the group
    - They can recognize, communicate, and/or interact with each other

Examples of Swarms

- Classic Example: Swarm of Bees
- Can be extended to other similar systems:
  - Ant colony

    Agents: ants
  - Flock of birds

    Agents: birds
  - School of Fish

    Agents: Fish
  - Robot Swarm

    Agents: Robot

Swarm Intelligence (SI)



- SI is the discipline that deals with artificial systems composed of many agents that coordinate using decentralized control and self-organization.
- Generally made up of agents who interact with each other and the environment.
- No centralized control structures.
- Based on group behavior.

SI System Properties

- It is composed of many individuals.
- The individuals are relatively homogeneous.
- The interactions among individuals are based on simple behavioral rules.
- The overall behavior of the system results from the interactions of individuals with each other and with their environment.

Common SI Algorithms



- **Particle Swarm Optimization**
- Ant Colony Optimization
- Bee Colony Optimization

Particle Swarm Optimization (PSO)

- The idea is similar to bird flocks searching for food.
  - Bird = Particle and Position of food = a solution



Particle Swarm Optimization (PSO) (cont.)



- PSO is a population-based, self-adaptive search optimization technique.
- It was developed in 1995 by James Kennedy and Russell Eberhart.
- It uses a number of agents (particles) that constitute a swarm moving around in the search space looking for the best solution.

Particle Swarm Optimization (PSO) (cont.)



- A swarm consists of *N* particles in a *D*-dimensional search space.
- Each particle holds
  - a position (which is a candidate solution to the problem) and
  - a velocity (which means the flying direction and step of the particle).

Particle Swarm Optimization (PSO) (cont.)

- Each particle successively adjust its position toward the global optimum based on two factors:
  - The best position visited by itself (*pbest*).
  - The best position visited by the whole swarm (*gbest*).
- Each particle moves forward by adjusting its "flying" according to
  - its own flying experience known as cognitive component as well as
  - the flying experience of other particles known as social component.

Personal Best (Cognitive behavior)

- Personal best position of a particle expresses the cognitive behavior of particle.
- It is defined as the best position found by the particle.
- It will be updated whenever the particle reaches a position with better fitness value than the fitness value of the previous personal best (pbest).

Global Best (Social behavior)

- Global best position expresses the social behavior.
- It is defined as the best position found by all the particles in the swarm.
- It will be updated whenever a particle reaches a position with better fitness value than the fitness value of the previous global best (gbest).

What a Particle Does?

- In each timestep, a particle has to move to a new position. It does this by adjusting its *velocity*.
  - *The current velocity PLUS*
  - *A weighted random portion in the direction of its personal best PLUS*
  - *A weighted random portion in the direction of the global best.*
- *Having worked out a new velocity, its position is simply its old position plus the new velocity.*

$$V_i(t+1) = V_i(t) + C_1.rand(\ldots).(P_i - X_i(t)) + C_2.rand(\ldots).(P_g - X_i(t))$$

$$X_i(t+1) = X_i(t) + V_i(t+1)$$

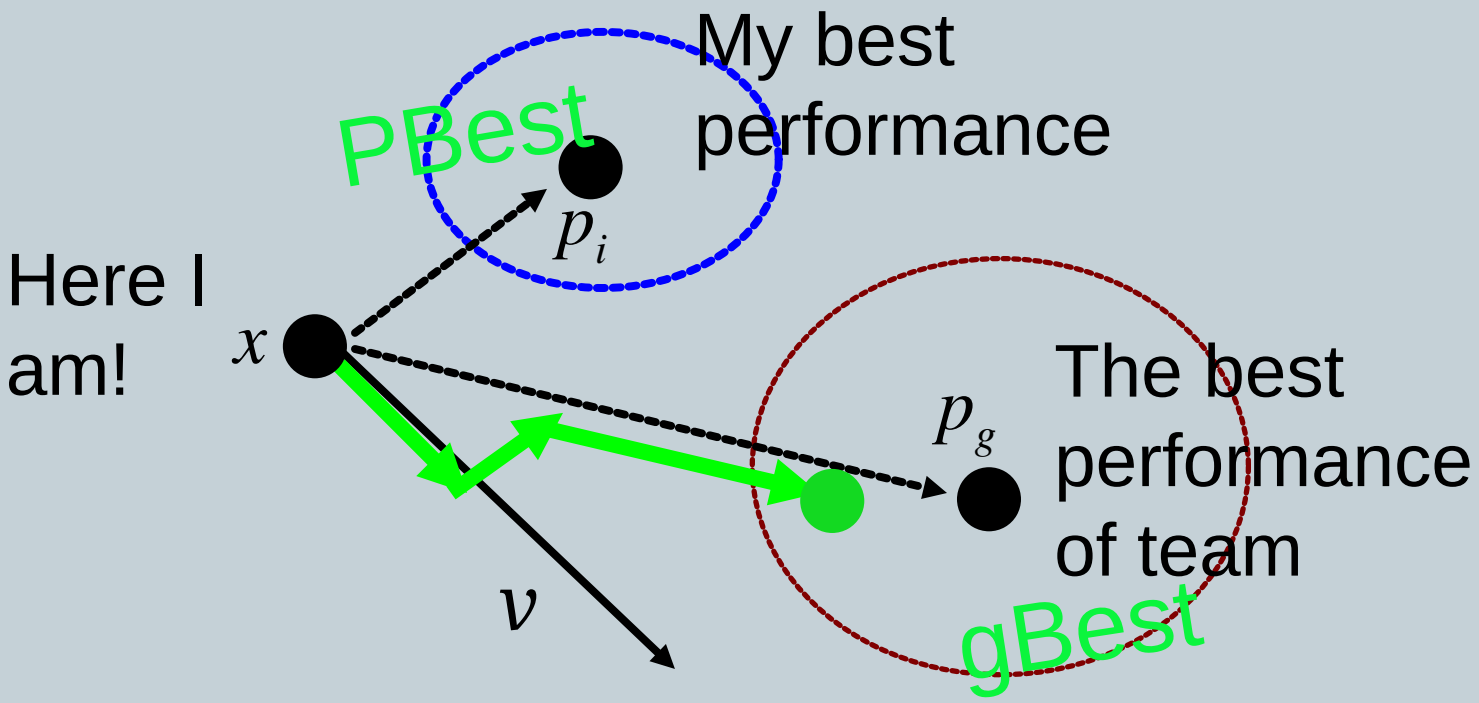
Parameters and Components

- C_1 and C_2 = acceleration coefficients. (Typically (C_1 + C_2) <= 4.
- rand(.) and rand(.) = random number between (0, 1).
- P_i –$X_i(t)$=“cognitive” component represents the personal thinking of each particle which encourages the particles to move their best positions found so far
- P_g -X_i(t)=“social” component which helps to find the global best position found so far.

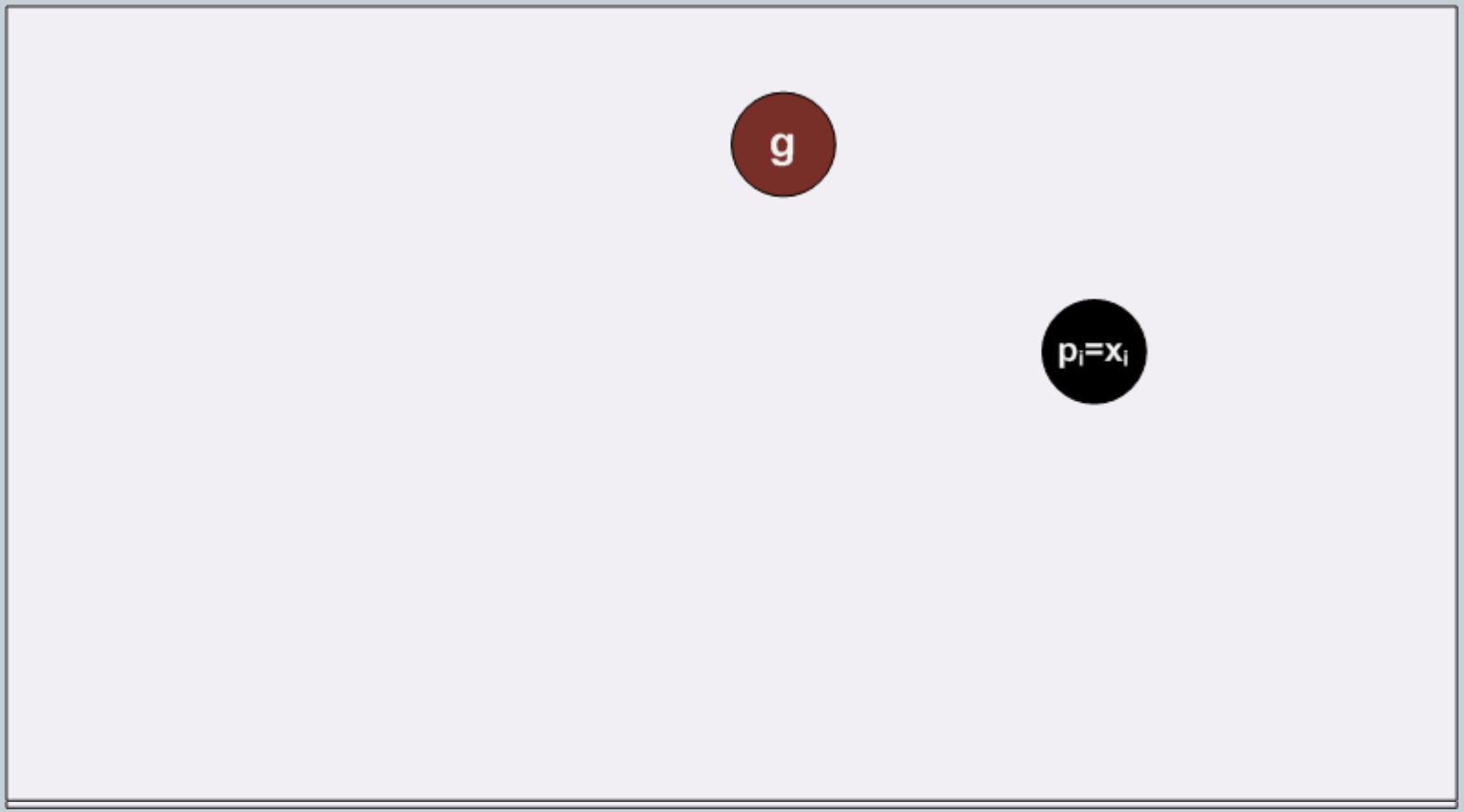
Single Particle

$$V_i(t+1) = V_i(t) + C_1.rand(\ldots).(P_i - X_i(t)) + C_2.rand(\ldots).(P_g - X_i(t))$$ **Eq.(1)**

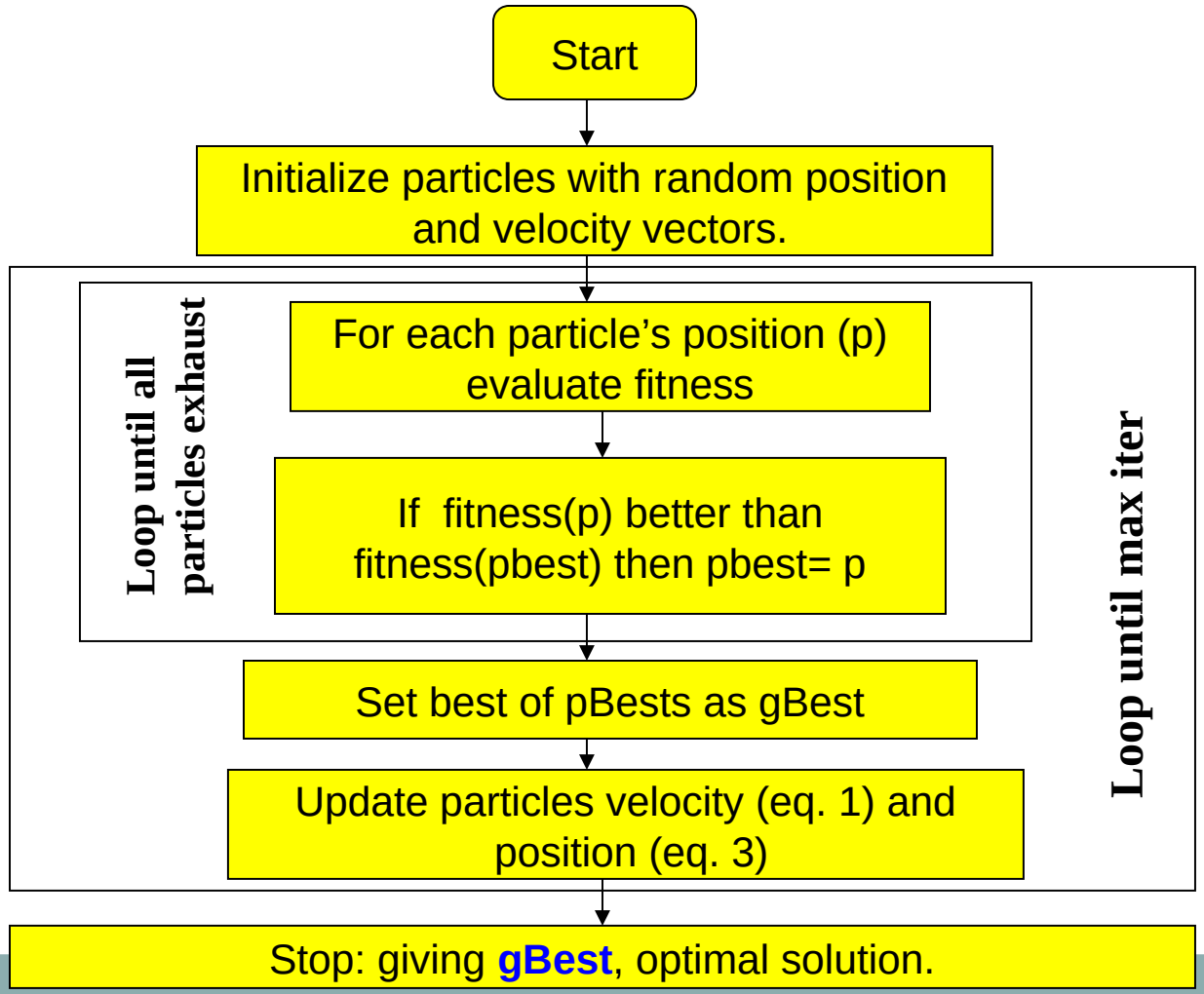
$$X_i(t+1) = X_i(t) + V_i(t+1)$$ **Eq.(3)**



Single Particle (Cont.)

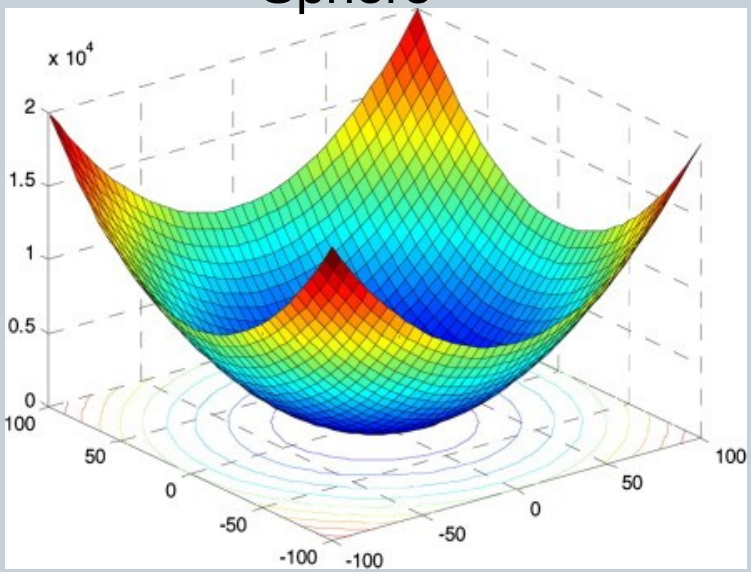


PSO Algorithm

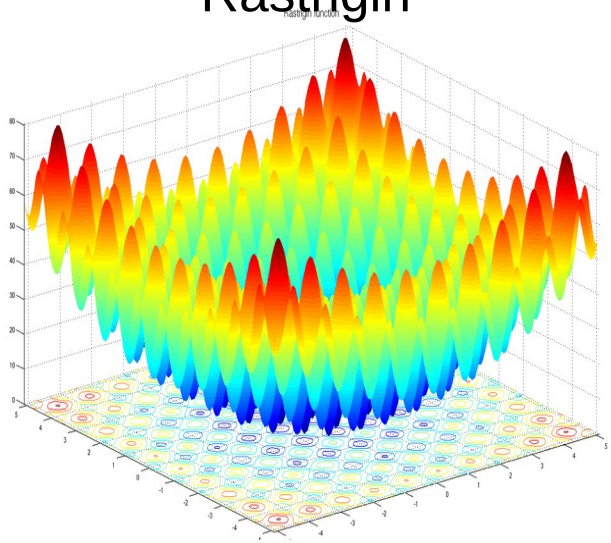


Benchmark Functions

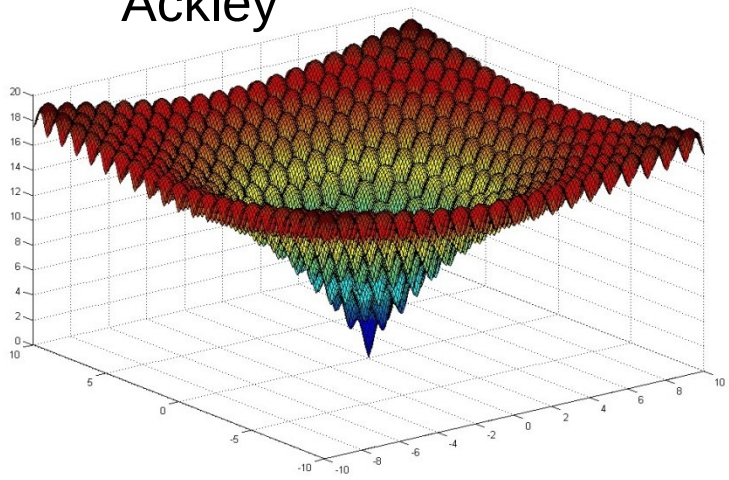
Sphere



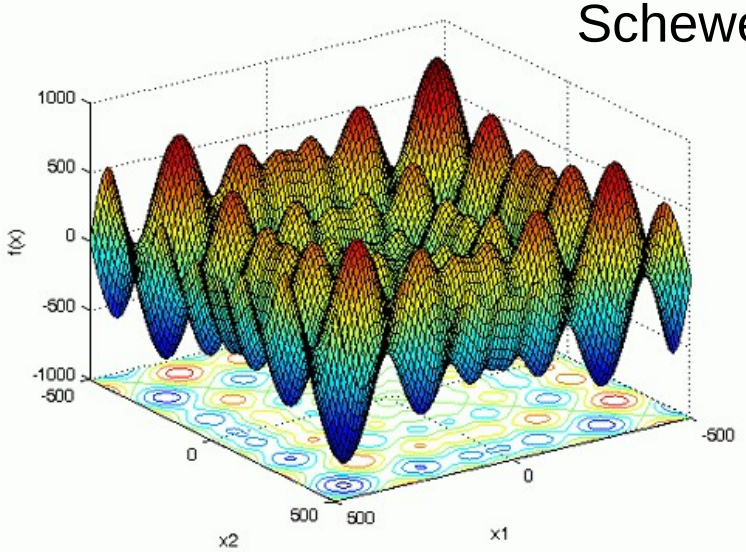
Rastrigin



Ackley



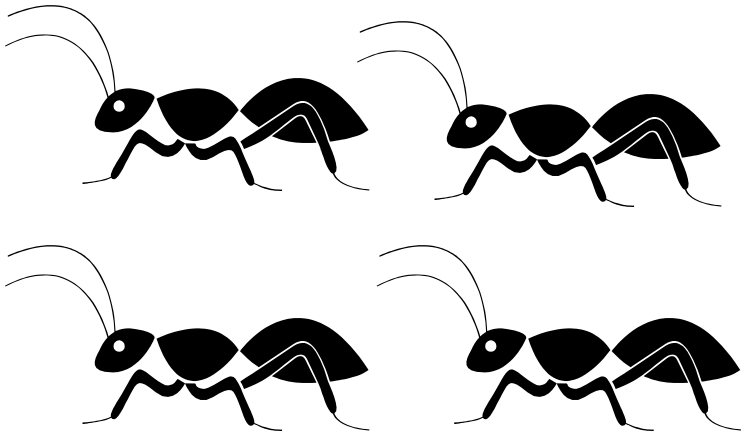
Schewefel



Problems in Multimodal Functions

- In the original PSO-
  - All particles learns from gBest in updating velocities and positions.
  - So, the algorithm exhibits a fast-converging behavior.
- But in multimodal problems,
  - A gBest located at a local optimum may trap the whole swarm and lead to premature convergence.

Ant Colony Optimization





Ant Colony Optimization

- A **probabilistic approach** for solving optimization problem.
- Proposed by **Marco Dorigo** in 1992 in his PhD thesis.
- Inspired by **behaviors** of real ants.

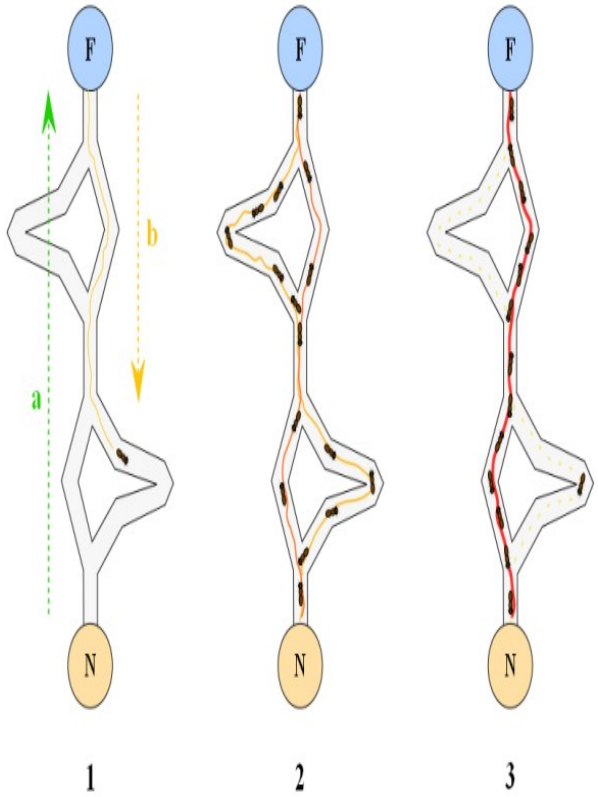


Natural Behavior of Ants

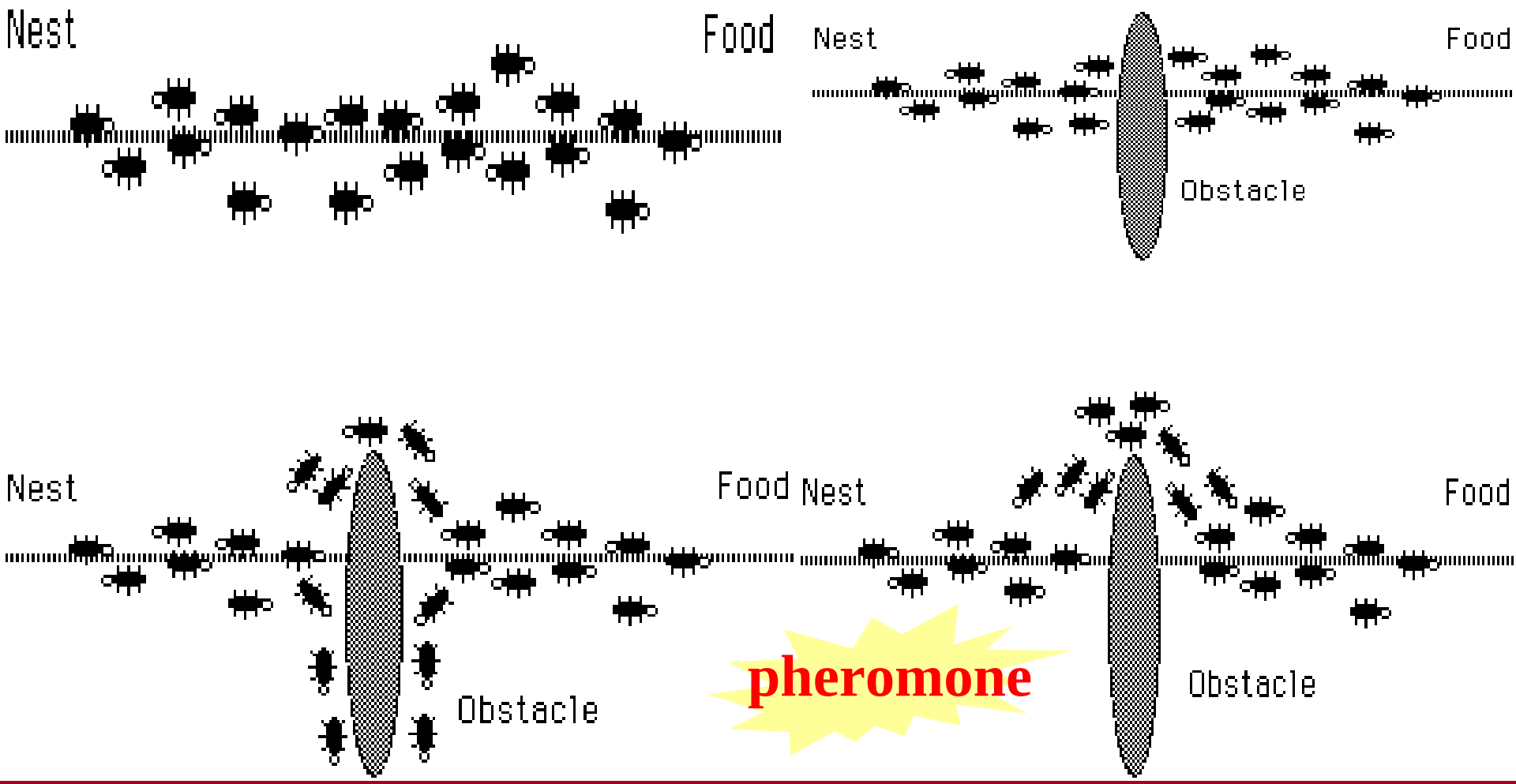
1. The first ant finds the food source (F), via any way (a), then returns to the nest (N), leaving behind a trail pheromone (b).

2. Ants indiscriminately follow four possible ways, but the strengthening of the runway makes it more attractive as the shortest route.

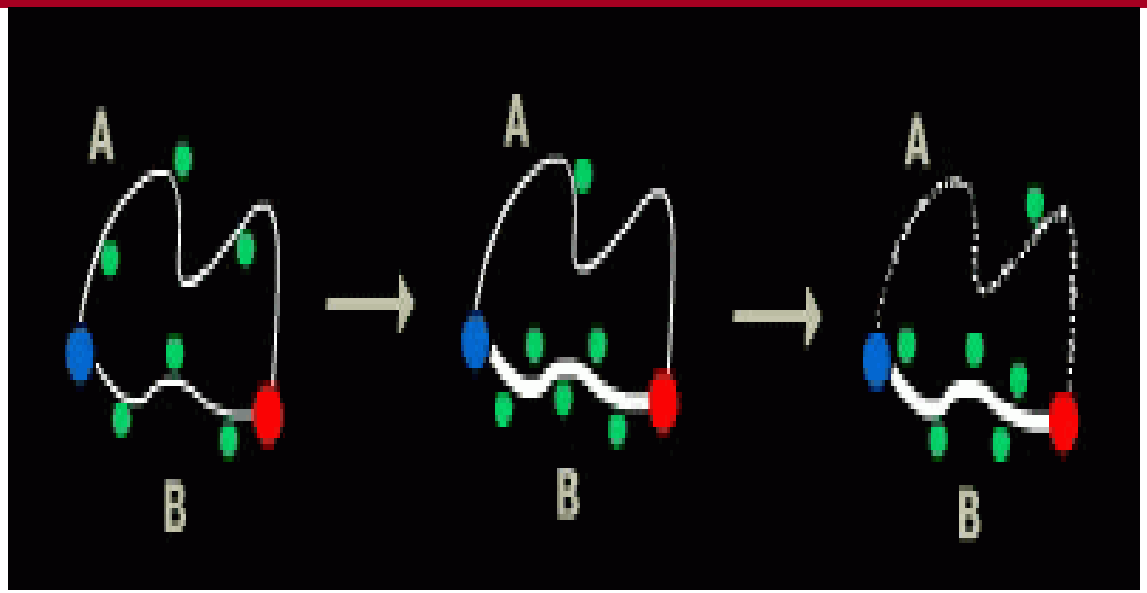
3. Ants take the shortest route; long portions of other ways lose their trail pheromones.



Natural ants: Finding the shortest path



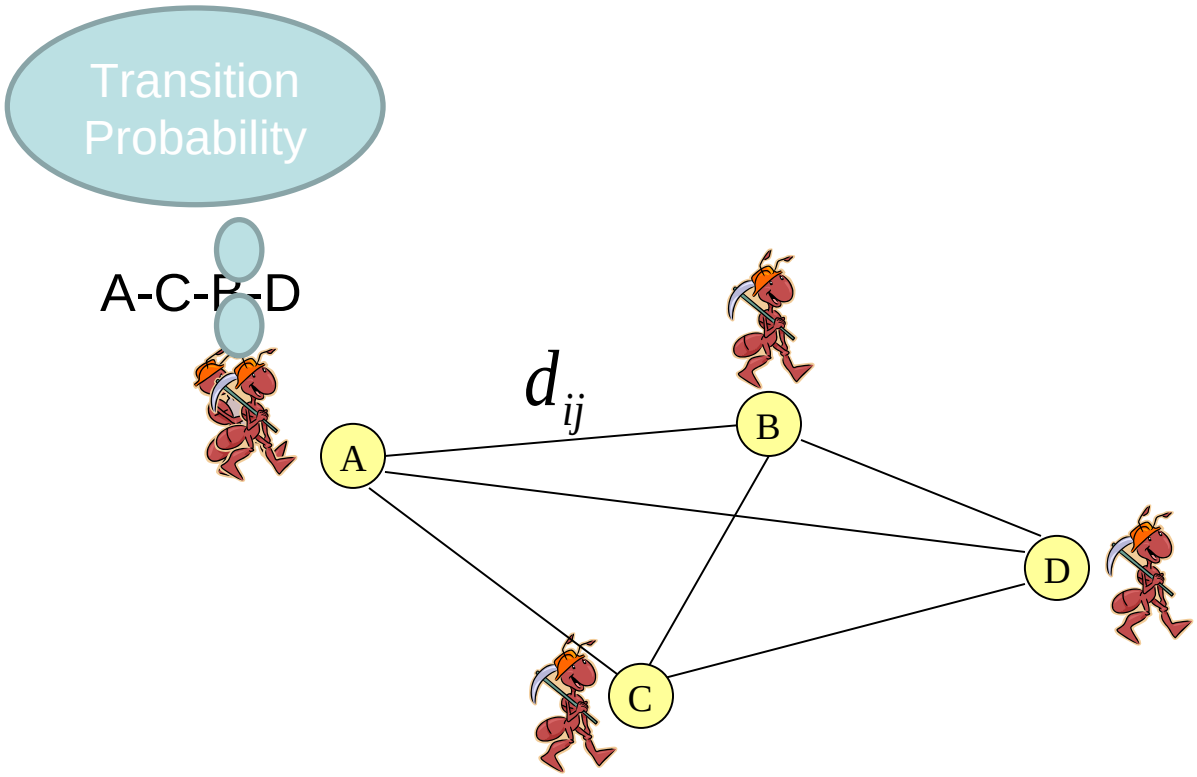
Natural ants: How do they do it?



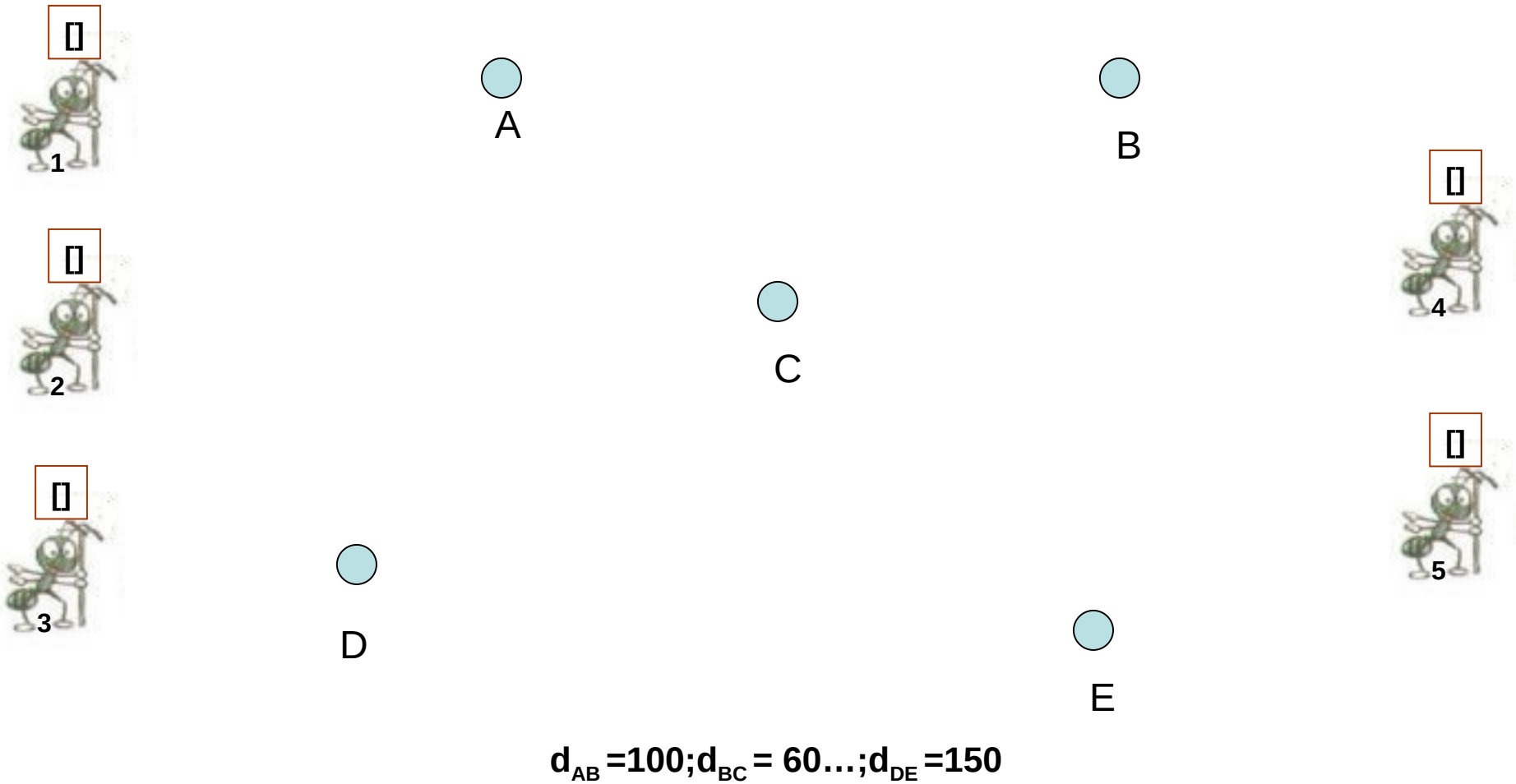
- Shortest path is discovered via **pheromone (evaporate quickly) trails**
  - each ant moves at random (first)
  - pheromone is deposited on path
  - Shorter path, more pheromone trails
  - ants follow the intense pheromone trails

Ant Colony Optimization

Graph (N,E): where N = cities(nodes), E = edges

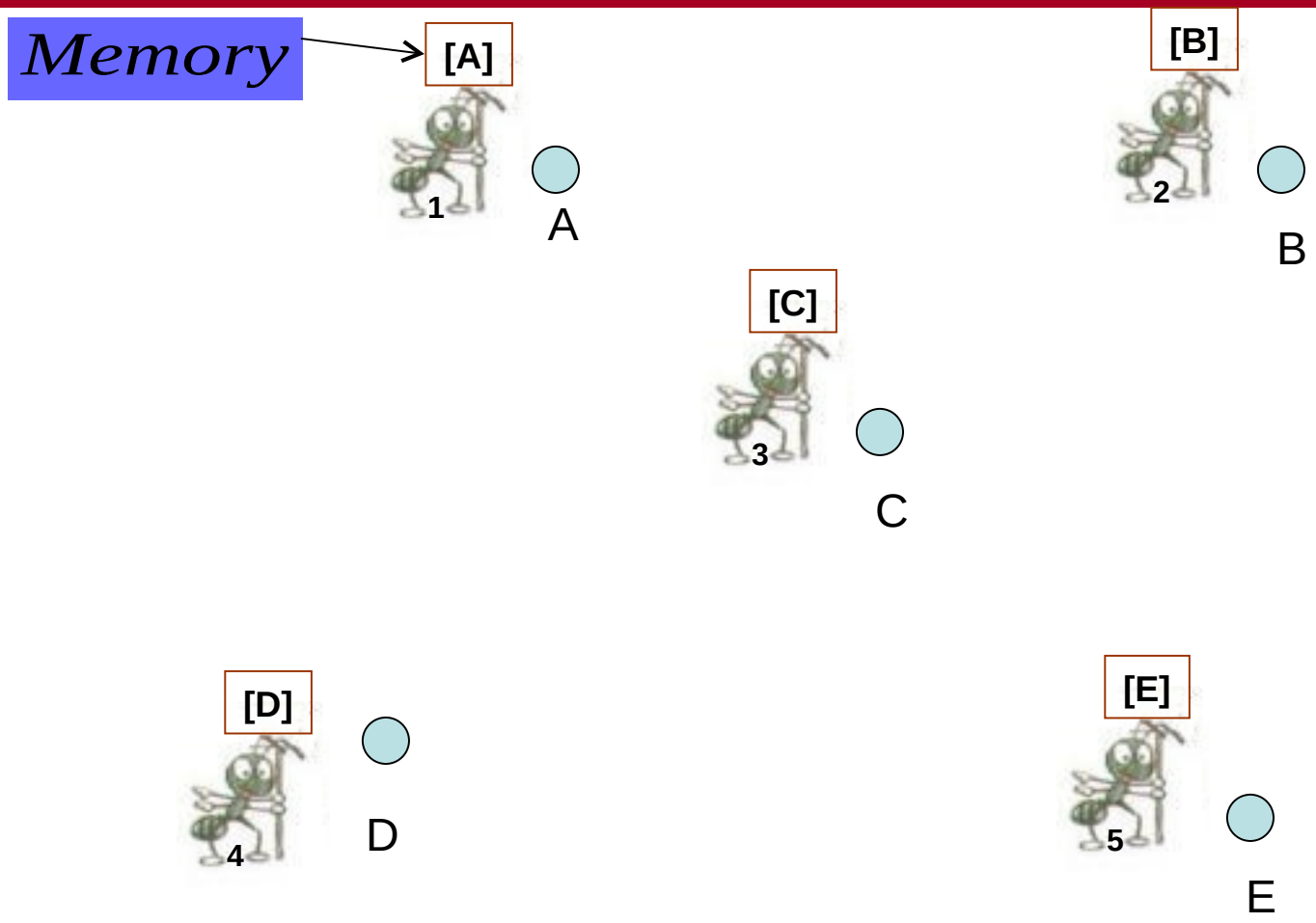


A simple TSP example

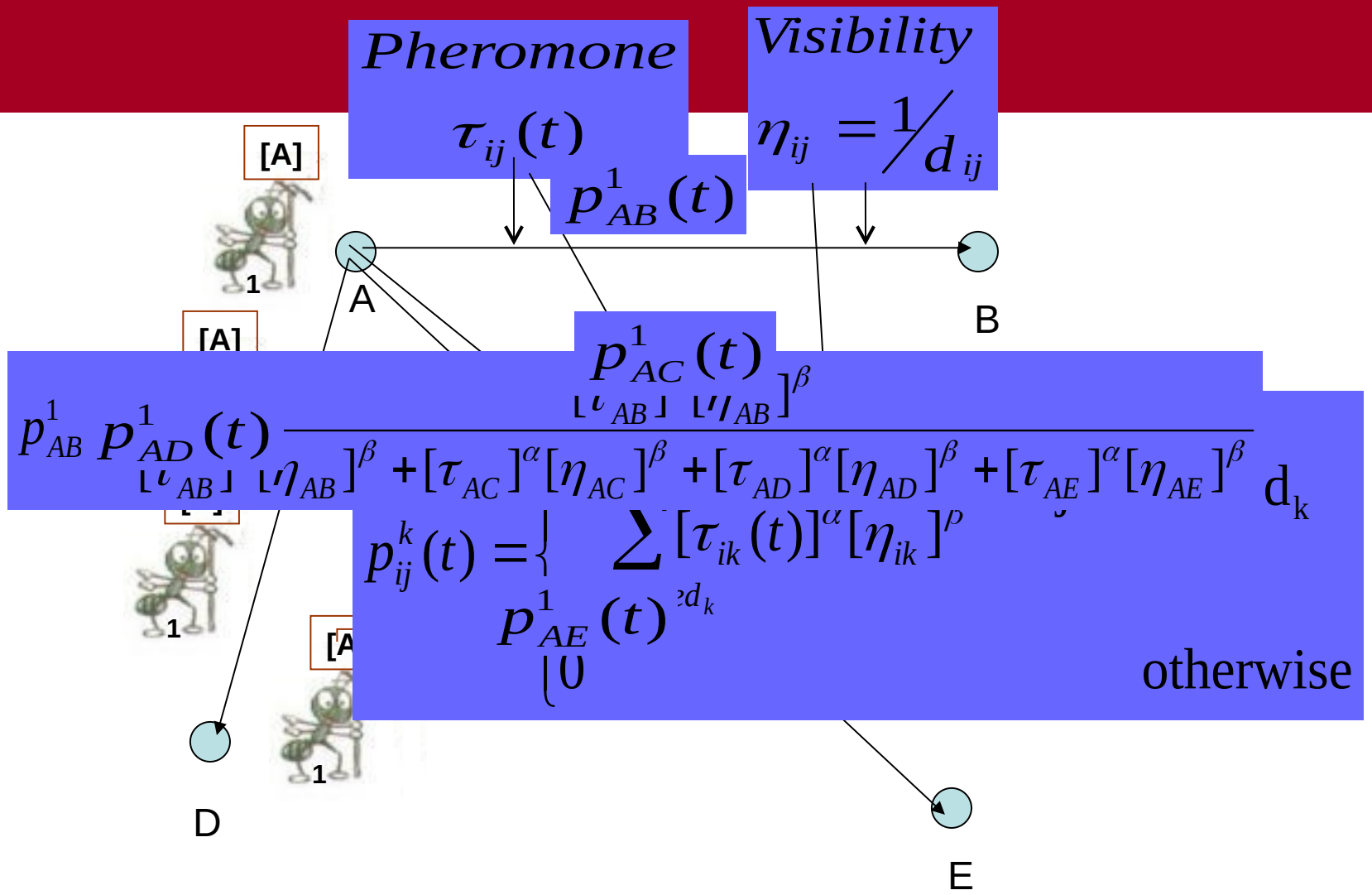


$d_{AB}=100; d_{BC}=60\ldots; d_{DE}=150$

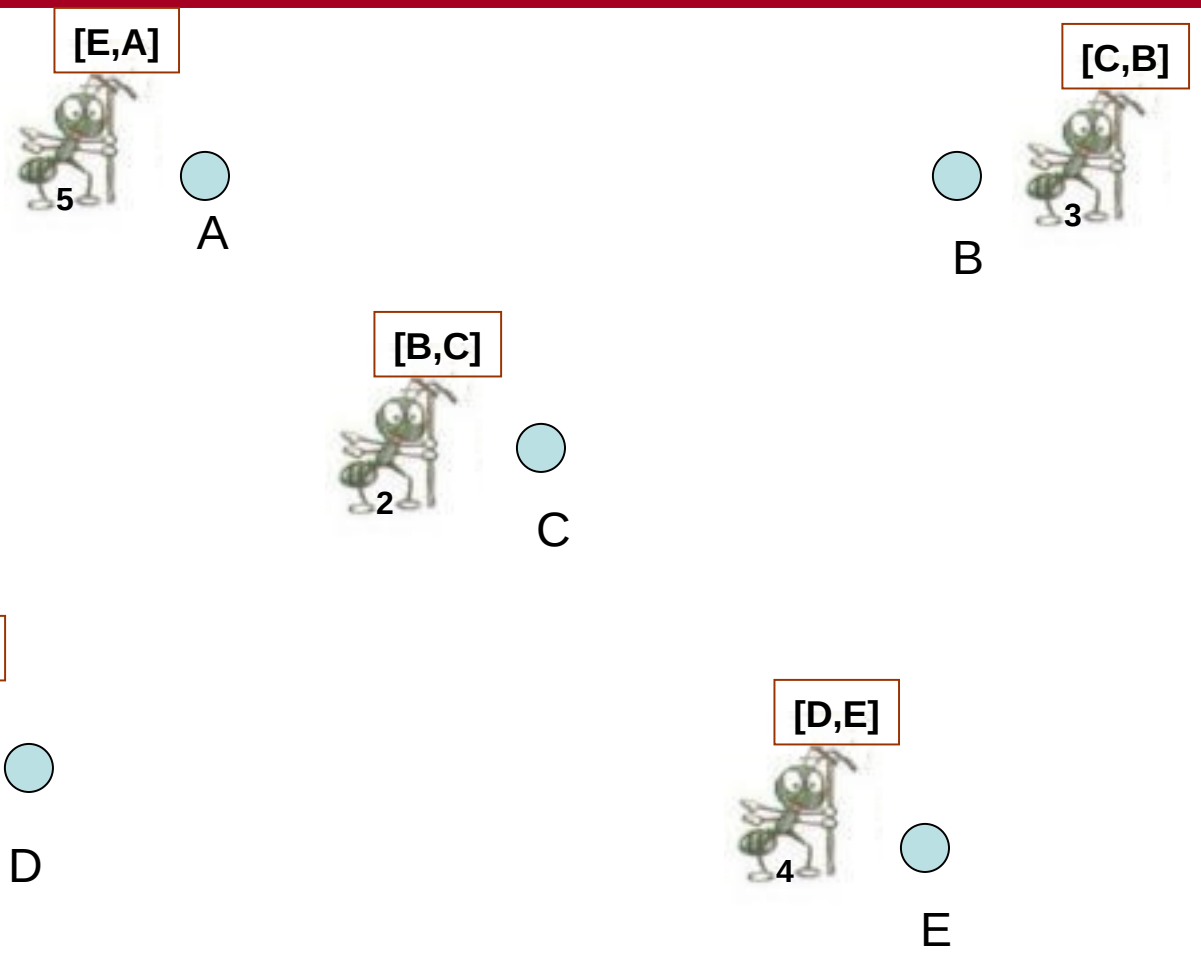
Iteration 1



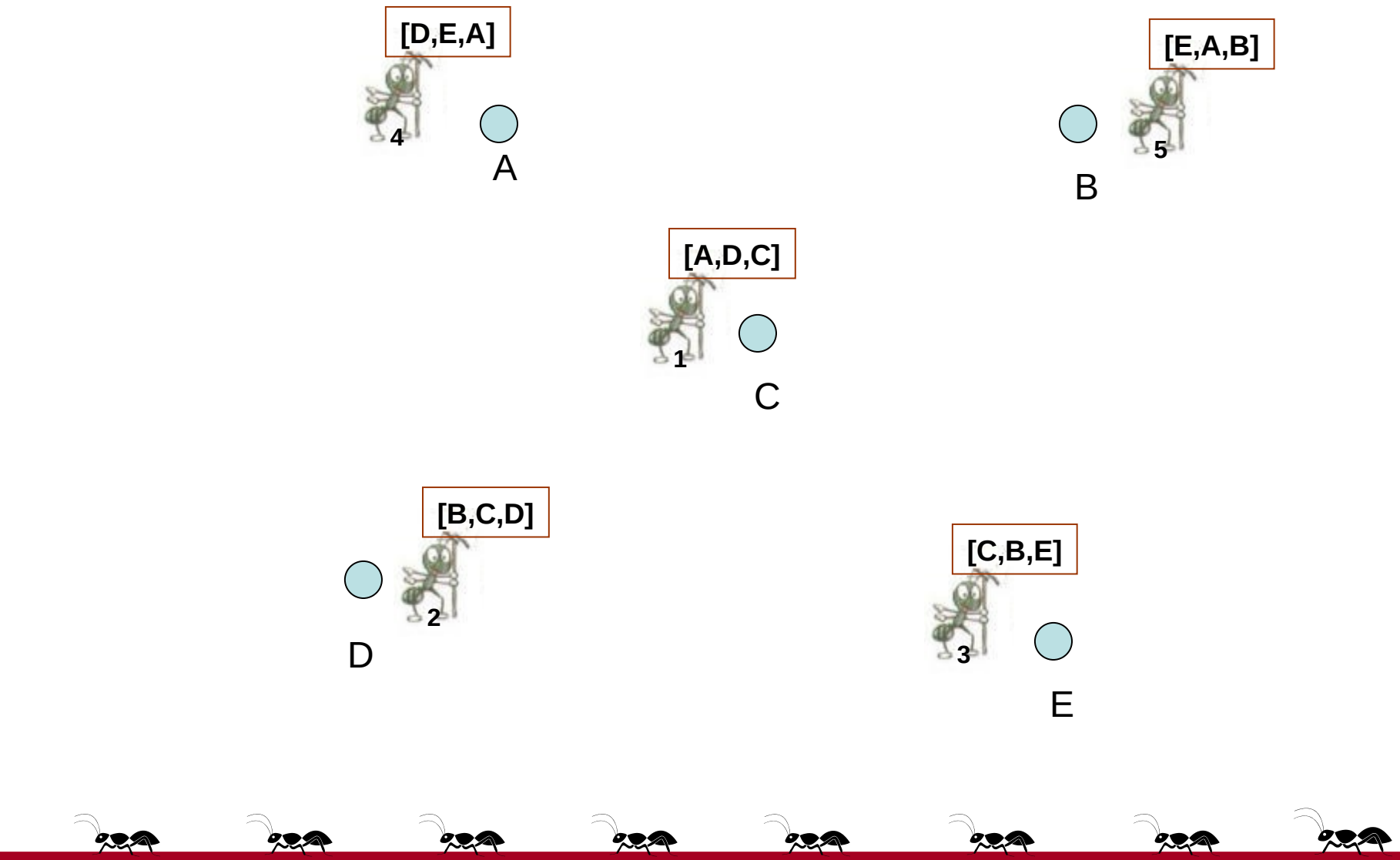
How to select next city



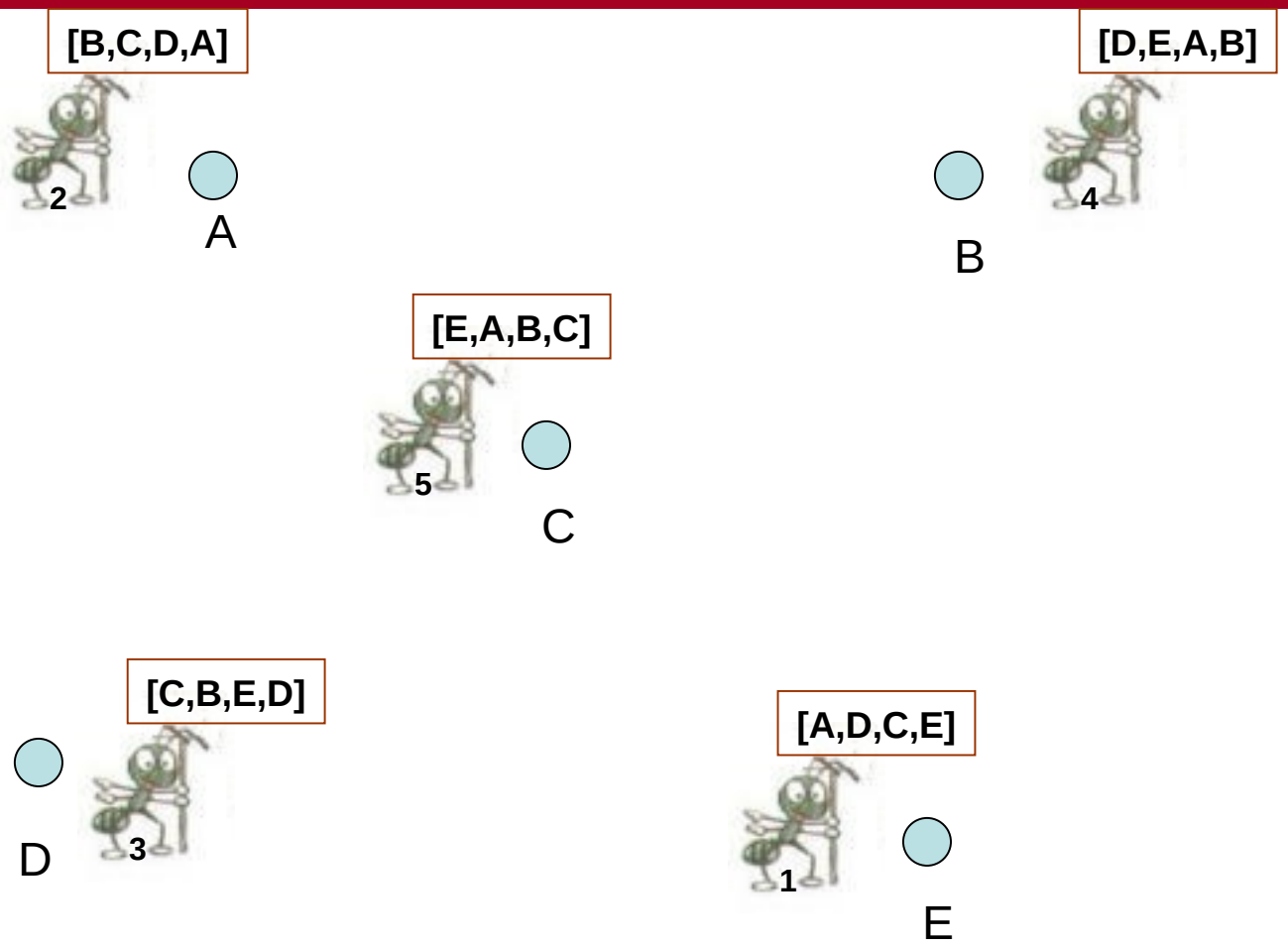
Iteration 2



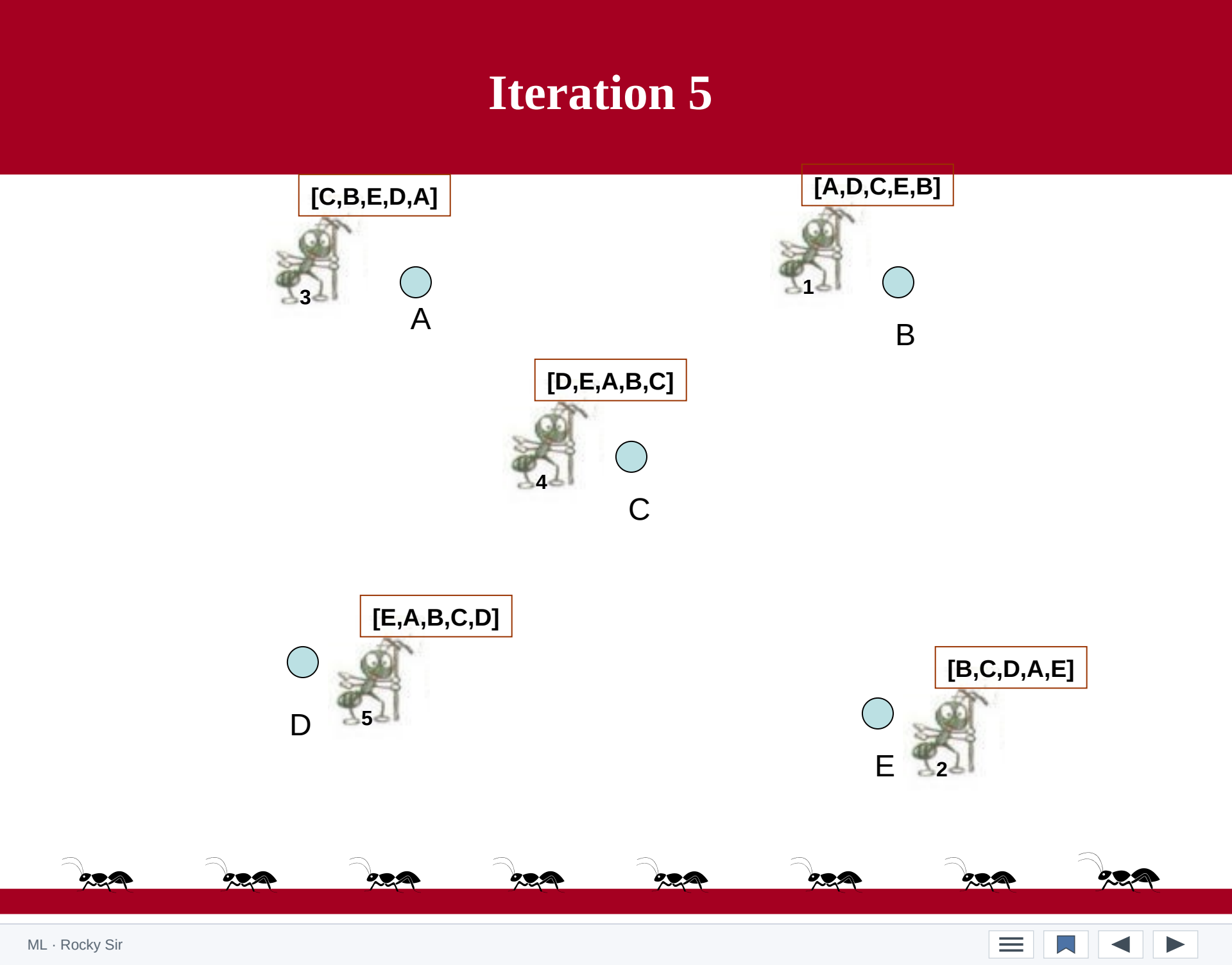
Iteration 3



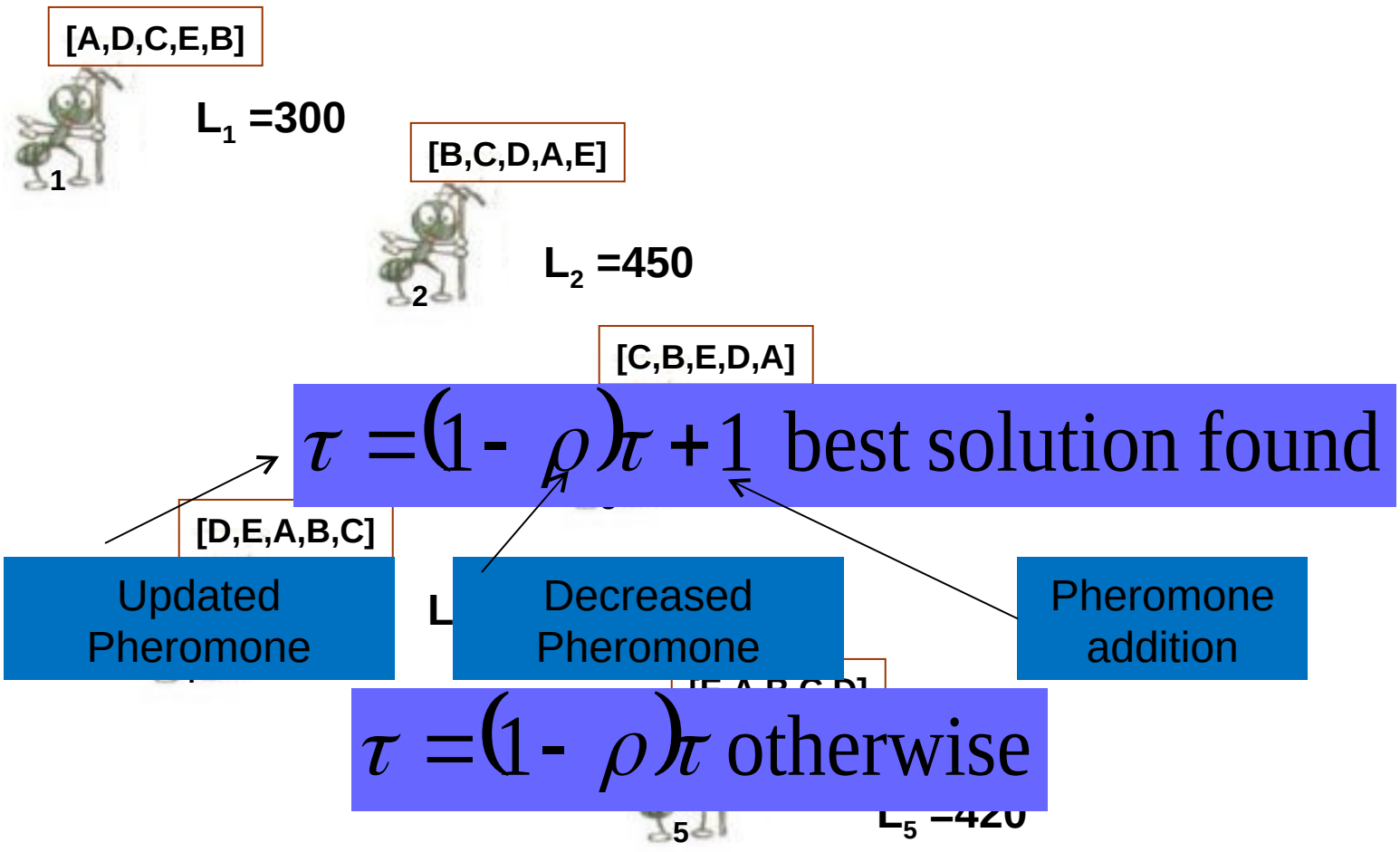
Iteration 4



Iteration 5



Pheromone Update



End of First Run

Save Best Tour (Sequence and length)

All ants die

New ants are born

Repeat this until all ants use the same tour
or
Maximum number of iteration

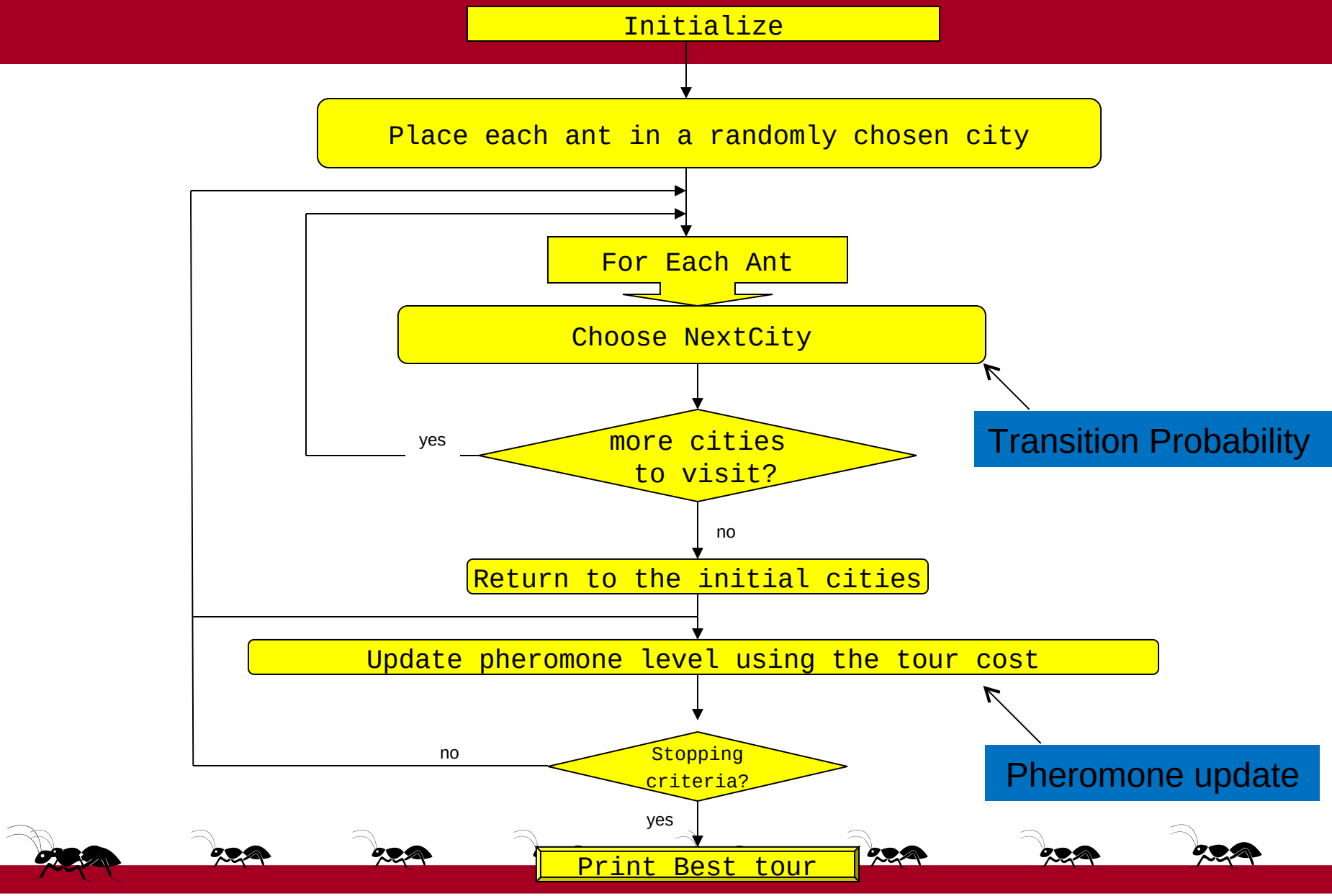


Parameters

- α: The relative importance of the trail, α>=0
- β: The relative importance of the visibility, β>=0
- 1-ρ: Trail evaporation, 0<=ρ<=1



Flow Chart: TSP



Applications

- Scheduling problem
- Telecommunication Network
- Wireless Sensor Network
- Vehicle routing problem



Advantages and Disadvantages

- Advantages:
  - Ants lay pheromone after completing tour so it works as **positive feedback** and it helps to discover good solutions rapidly.
  - **Distributed computation** avoids premature convergences.
- Disadvantages:
  - **Slower convergence** than other Heuristics.
  - **Performs poorly** for TSP problems larger than 75 cities**.**



Conclusion

- The optimal number of ants is close to the number of cities ($m \approx n$)
  - m=number of ants, n=number of cities
- Default value of the parameters:
  - $\alpha = 1$, $\beta = 1$, $\rho = 0.5$



References

- [Dorigo et al., 1996]
  - Dorigo M., V. Maniezzo & A. Colorni (1996). The Ant System: Optimization by a Colony of Cooperating Agents. *IEEE Transactions on Systems, Man, and Cybernetics-Part B, 26(1):29-41*
- [Dorigo & Gambardella, 1997]
  - Dorigo M. & L.M. Gambardella (1997). Ant Colonies for the Traveling Salesman Problem. *BioSystems, 43:73-81.*
- http://en.wikipedia.org/wiki/Ant_colony_optimization_algorithms
- http://en.wikipedia.org/wiki/Heuristic
- http://iridia.ulb.ac.be/~mdorigo/ACO/ACO.html
- Book- “Ant Colony Optimization”, Dorigo and Stutzle
- *www.divms.uiowa.edu/~hzhang/c145/notes/12-AntColony.ppt*
- *140.133.35.1/faculty/pwu/heuristic/ACS_1.ppt*

Artificial Bee Colony (ABC) Algorithm

Artificial Bee Colony (ABC) (History)

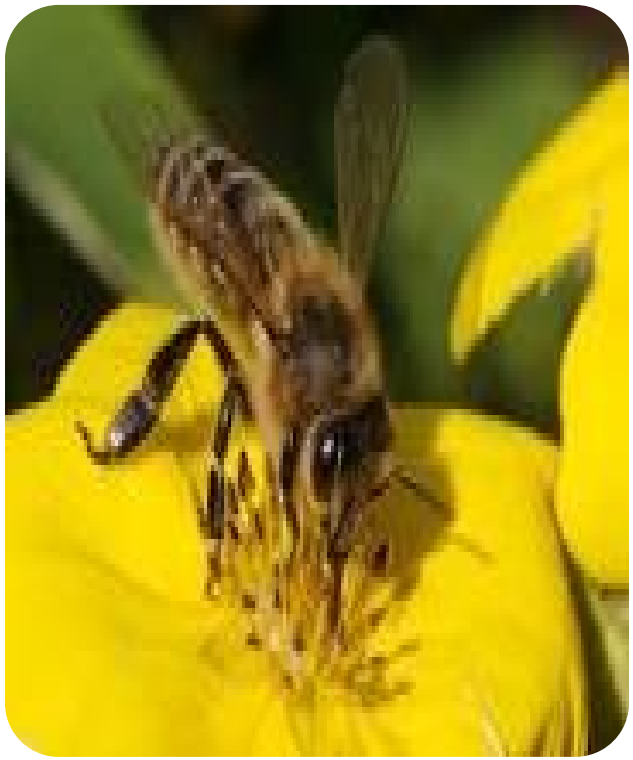
- Artificial Bee Colony (ABC) is one of the most recently defined algorithms by Dervis Karaboga in 2005, motivated by the intelligent behavior of honey bees.



Artificial Bee Colony

- The ABC algorithm is a swarm based meta-heuristics algorithm.
- It based on the foraging behavior of honey bee colonies.
- The artificial bee colony contains three groups of bee:

➢ Employed bees
➢ Onlookers
➢ Scouts

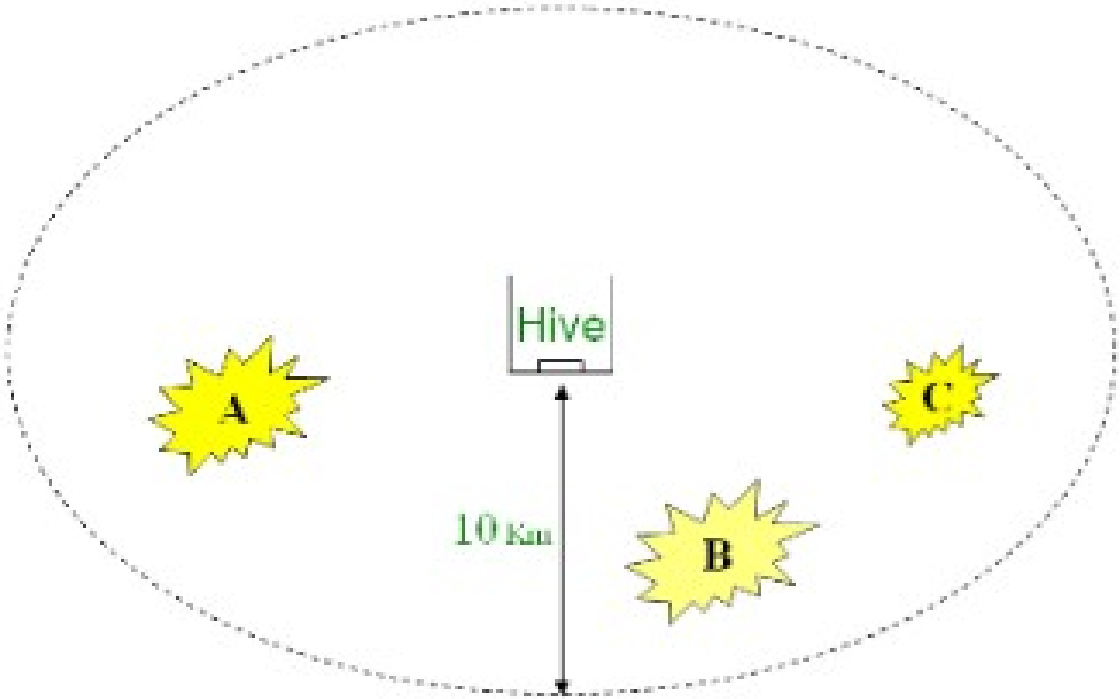


Artificial Bee Colony (ABC)

- The Employed Bee:
  It stays on a food source and provides the neighborhood of the source in its memory.
- The Onlooker Bee:
  It gets the information of food sources from the employed bees in the hive and select food sources to gathers the nectar.
- The Scout:
  The employed bee whose food source has been abandoned becomes a scout and starts to search for finding a new food source.

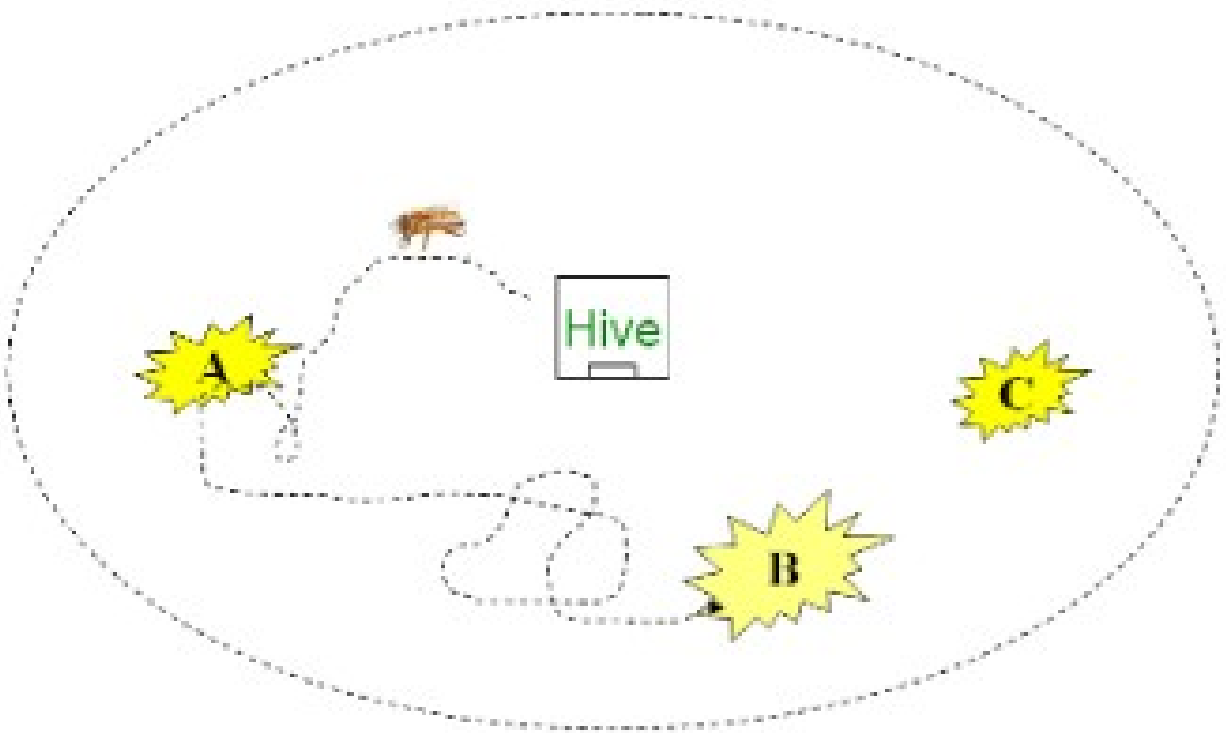
Bees in Nature

- A colony of honey bees can extend itself over long distances in multiple directions (more than 10 km).



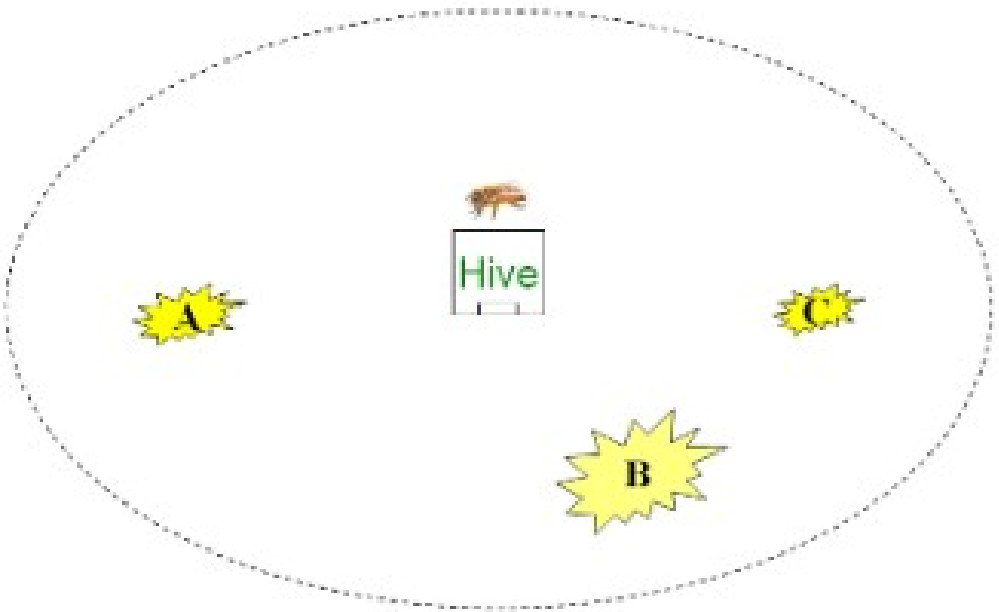
Bees in Nature

- Employed bees go to their food source.



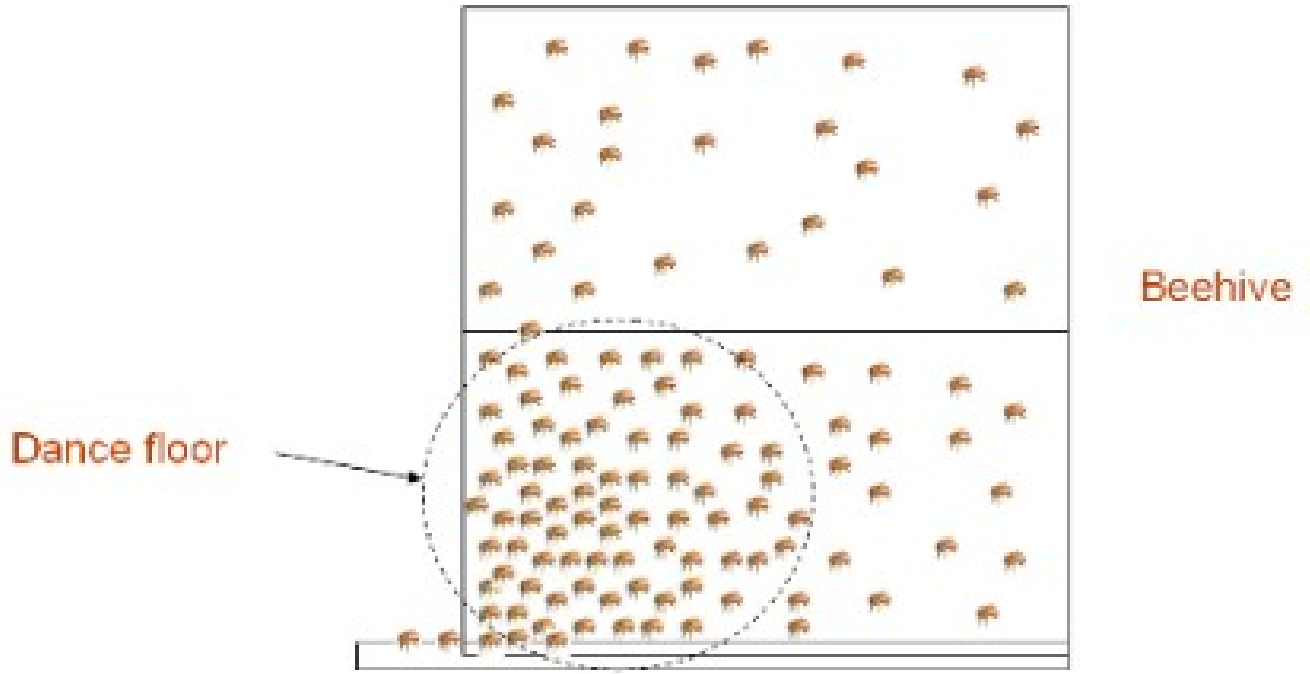
Bees in Nature

- The employed bees who return to the hive, evaluate the different patches depending on certain quality threshold (measured as a combination of some elements, such as sugar content).



Bees in Nature

- They go to the “dance floor” to perform a “waggle dance”



Bees in Nature



Bees communicate through this waggle dance which contains the following information:

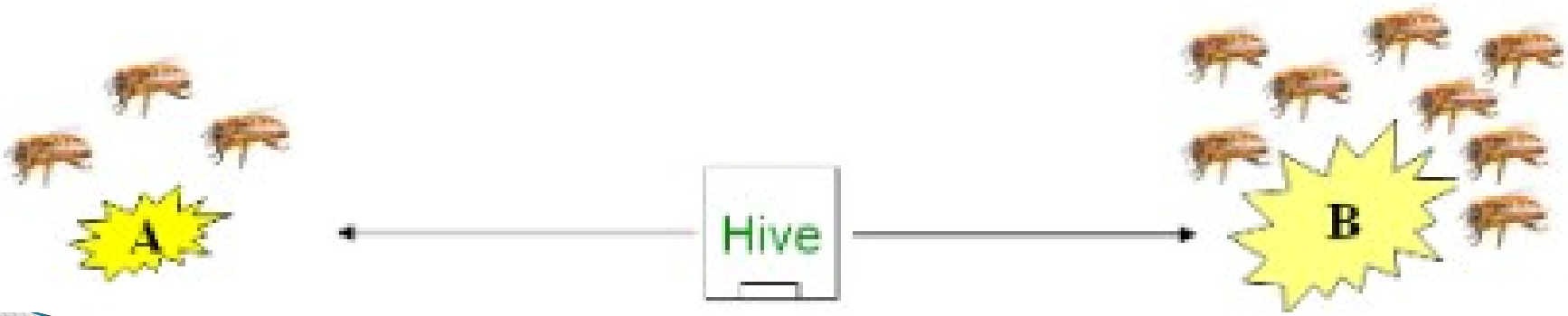
1. The **direction** of flower patches (angle between the sun and the flower patch)
2. The **distance** from the hive (duration of the dance)
3. The **quality** rating (fitness) (frequency of the dance)

Bees in Nature

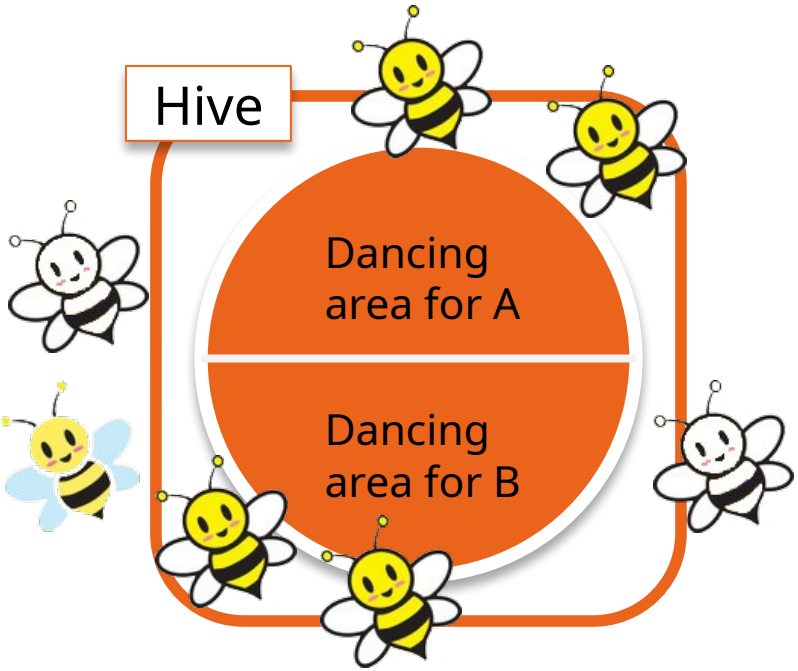
- These information helps the colony to send its bees precisely
- Follower bees (onlooker bees) go after the dancer bee (employed bees) to the patch to gather food efficiently and quickly

Bees in Nature

- Thus, according to the fitness, patches can be visited by more bees or may be abandoned
- The employed bee whose food source has been abandoned becomes a scout and starts to search for finding a new food source



Bees in Nature



Picture form www.acclaimclipart.com , www.computerclipart.com

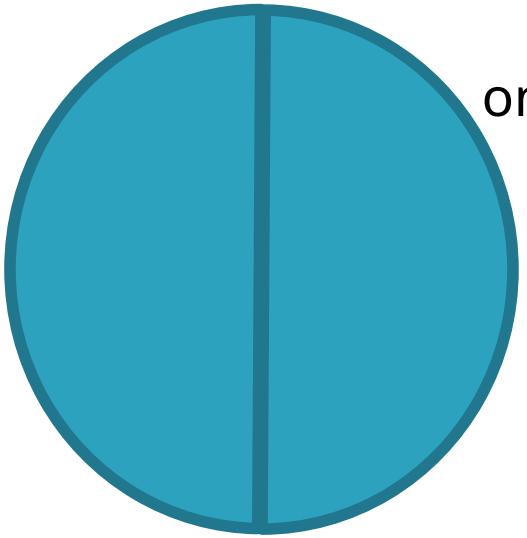
ABC Algorithm

- Step 1: Assign control parameters
- Step 2: Initialize solutions and calculate fitness (employed bees)
- Step 3: Repeat until stopping criteria is met
  - employed bees generated neighborhood position and calculate fitness
  - onlooker bees generated position and calculate fitness
  - scout bees generated position and calculate fitness
  - memorize the best solution
- Step 4: Stop

ABC Algorithm

Foraging bee

employed



onlooker

Scout

ABC control parameters

- Swarm size (Total solutions)
- Employed bees (50% of swarm)
- Onlookers (50% of swarm)
- Scouts (1 or can be more than 1)
- Limit (for becoming a scout from an employed bee)
- Dimension of the problem



Initial Solutions

Food source initialization
(Number of solutions = Employed bees)

$$x_{i,j} = x_{\min,j} + rand(0,1)(x_{\max,j} - x_{\min,j})$$

Where,

i = 1,2,…,n

n = Food source

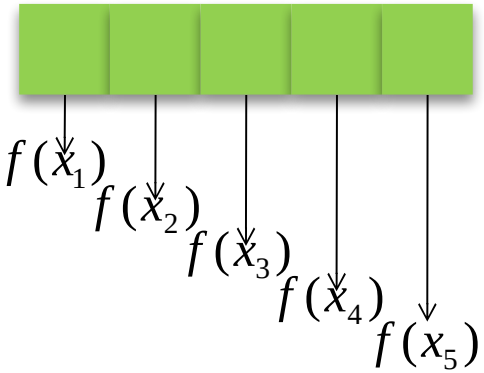
j = Dimension

Employed bee Phase

- Send each employed bee to solutions/positions of flowers (Initial Solutions)
  - Number of solutions = Employed bees = 5 (in this example)
  - 1 dimension (in this example)



- Calculate fitness

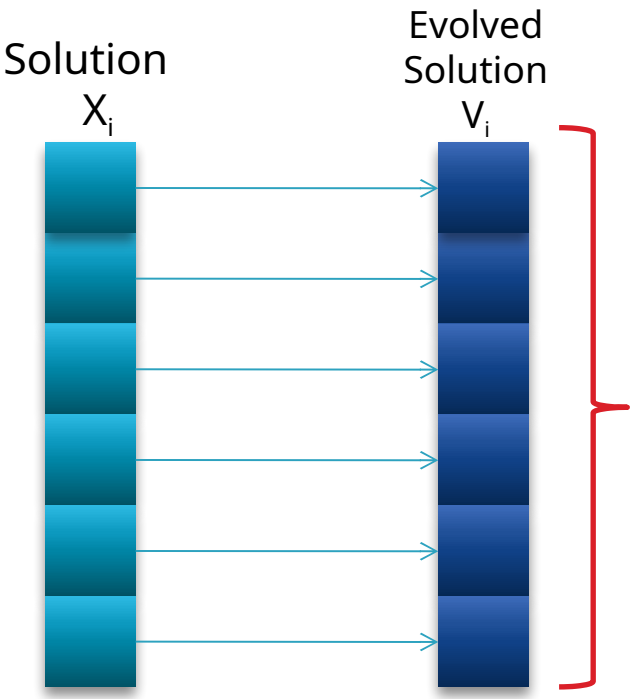


Employed bee Phase (Cont.)

- Evolve solution to neighborhood

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj})$$

Where,
ϕ_{ij}= rand(-1, 1)
i = 1,2,...,n
n = Food source
j = Dimension
k = rand(1, n) != i



j=6

- Single solution evolution

- Select better solution comparing *f(xi)* and *f(vi)*

Onlookers Phase

- Calculate probability for each solution

$$P\{x_i\} = \frac{f(x_i)}{\sum_{i=1}^{n} f(x_i)}$$

- Select solution with respect to probability
- Can use roulette wheel or other technique to make probabilistic nature

Onlookers Phase (cont.)

- Evolve selected solution

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj})$$

Where,
ϕ_{ij}= rand(-1, 1)
i = 1,2,...,n
n = Food source
j = Dimension
k = rand(1, n) != i

- Calculate *f(vi)*
- Select better solution comparing *f(xi)* and *f(vi)*

(Same as Employed bee phase)

Scout Phase



- In ABC, providing that a position can not be improved further through a predetermined number of cycles, which is called "limit" then that food source is assumed to be abandoned.

-some employed bees (food source abandoned) may become scouts to explore the search space by the following equation (same as initialization)

$$x_i^j = x_{\min}^j + \text{rand}(0,1)(x_{\max}^j - x_{\min}^j)$$

Advantages & Disadvantages

- Advantages
  - Few control parameters
  - Fast convergence
  - Both exploration & exploitation
- Disadvantages
  - Search space limited by initial solution

Applications

- Mechanical designs like:
  - Design of welded beam
  - Design of coil spring
- Digital Filter Optimisation
- Fuzzy Control Design
- Data Clustering (solving the local optimum for K-means algorithm)
- Robot control